

修士論文

バードストライク検出に向けた
コンピュータビジョンと
UAV の活用

山形大学大学院
理工学研究科数理科学分野
23321427 千葉歩夢

2025 年 2 月

概要

近年、再生可能エネルギーの 1 つである風力発電の導入が注目されているが、発電機の建設には多くの問題点が存在する。その中でもバードストライクは重大な問題であり、回転する風車のブレードに鳥類が衝突する現象である。山形県企業局では、当局が管理している風力発電機に対してバードストライク調査を行っているが、仕事量の多さや調査の正確性に幅があることが懸念されている。本研究の目的は、この懸念を解消するために、ドローンを用いて発電機周辺の土地を撮影し、自動的に鳥を発見できるプログラムを構築することである。

昨年度、山形大学の先輩である梅津魁秀さんは、深層学習を用いたバードストライク調査プログラムの研究を行い、AE を使用してドローンで撮影した画像から物体画像を検出する手法を提案し、物体画像に関して F2-score が 0.83 という高い精度で物体の有無を判定することができた。

今年度は課題であった、汎用的なモデルを構築することと物体画像から鳥の画像を検出することに取り組んだ。汎用的なモデルの構築には、モデルの自動アップデートを行い、物体画像から鳥の画像を検出するために CLIP の使用を提案した。

本研究の成果により、AE, PCA, K-means クラスタリング, CLIP を組み合わせることで自動アップデートの手法を開発したが、アップデート後の F2-score が 0.15 と低く、全ての物体の検出も出来ていないため汎用的なモデルを構築することは出来なかった。これは、アップデート元となるモデルが過剰にデータ拡張していたためと考えられる。

物体画像から鳥の画像の検出は、CLIP を用いて可能であることが確認できた。しかし、手動でテストデータを構築したため、厳密な評価を行うのは難しいが F2-score は 0.33 であった。

今後の課題として、過剰なデータ拡張を行っていないモデルをアップデート元とすることと、検出できた物体画像に対して CLIP を用いて鳥の検出の有無を評価することが挙げられる。

目次

1	はじめに	2
1.1	背景と目的	2
1.2	先行研究の概要	2
1.3	引き継ぎの理由と目標	2
2	検出方法とアルゴリズム	3
2.1	科学用語の説明	3
2.2	検出方法	7
2.3	閾値に関して	8
2.4	AE の改良	9
2.5	AE モデルのアップデートの重要性	12
2.6	モデルのアップデート方法	13
3	実験	14
3.1	物体検出の試み	14
3.2	モデルアップデートの有用性	15
3.3	背景画像の分類	15
3.4	物体画像の抽出	19
3.5	モデルのアップデート	23
3.6	物体検出の試み	24
3.7	物体検出システムの確認	26
3.8	差分画像の精度向上	32
3.9	物体から鳥を抽出	36
4	結論	37
5	謝辞	37
6	使用したデータの説明	38
6.1	プログラミング環境と使用したライブラリ	38
6.2	想定環境	39
6.3	画像データ	39
6.4	モデルデータ	40
6.5	使用プログラム	41
6.6	モデル構築	45
6.7	データ	72

1 はじめに

1.1 背景と目的

近年, 再生可能エネルギーの 1 つである風力発電の導入が注目されているが, 発電機の建設には問題点も多い. バードストライクはその問題点の 1 つであり, 回転する風車のブレードに鳥類が衝突する現象である. 山形県企業局では, 当局が管理している風力発電機に対してバードストライク調査を行っているが, 仕事量の多さや正確性に幅があることが懸念されている. この懸念を解消するために, ドローンを用いて発電機周辺の土地を撮影し, 自動的に鳥を発見できるプログラムを構築することが本研究の目的である.

詳しい内容は昨年度の論文 [1] で詳細に説明しているので, そちらを参照して欲しい.

1.2 先行研究の概要

昨年度, 山形大学の先輩である梅津魁秀さんは, 深層学習を用いたバードストライク調査プログラムの基礎的な研究を行った. 彼の研究では, AE を用いて物体検出モデルを構築し, ドローン画像から鳥を検出する手法を提案した. 具体的には, ドローンで撮影した画像を「物体が存在する画像」と「物体が存在しない画像」に分類し, 「物体が存在する画像」と GUI を組み合わせることで手動で鳥を検出する方法を開発した. 彼の研究により, 物体画像に関して F2-score が 0.83 という高い精度で物体の有無を判定することができた.

1.3 引き継ぎの理由と目標

梅津氏の研究は有望な結果を示したが, 実用化にはさらなる改良が必要である. 特に, 昨年度は限られた画像において高精度な結果を残したため, 汎用的なモデルを構築することと, 物体画像から鳥の画像を自動で検出することが挙げられる. 本研究では, これらの課題に対してモデルの自動アップデートと CLIP を用いる手法を用いて課題解決にアプローチした.

2 検出方法とアルゴリズム

本章では、検出方法やアルゴリズムを説明する。なお、今年度は引継ぎのため重複する部分は省略し、新規の内容や変更した内容のみを説明する。

2.1 科学用語の説明

本研究では、目標達成のためにさまざまな技術を使用している。以下に、研究の中で使用される主要な科学用語を説明する。

AE (Auto Encoder)

AE は、入力データを圧縮し、その圧縮されたデータから元のデータを再構築することを目的としたニューラルネットワークの一種である。主にデータの次元削減や特徴抽出に使用される。AE はエンコーダとデコーダの 2 つの部分から構成され、エンコーダは入力データを低次元の潜在空間にマッピングし、デコーダはその潜在空間から元のデータを再構築する。

ファインチューニング (Fine-Tuning)

ファインチューニングは、事前に学習されたモデルを新しいデータセットに適応させるための手法である。これにより、モデルの性能を向上させることができる。ファインチューニングでは、通常、事前学習されたモデルの一部または全ての層の重みを微調整する。

CLIP (Contrastive Language-Image Pre-Training)

CLIP は、OpenAI によって開発されたモデルで、画像とテキストのペアを用いて学習することで、画像とテキストの間の関連性を高精度で捉えることができる。CLIP は、画像エンコーダとテキストエンコーダを用いて、それぞれの入力を高次元のベクトルに変換し、これらのベクトル間のコサイン類似度を最大化するように学習する。これにより、画像とテキストの間の関連性を高精度で捉えることができる。

ResNet (Residual Networks)

ResNet は、深層ニューラルネットワークの一種であり、深いネットワークにおける勾配消失問題を解決するために開発された。ResNet は、残差ブロックを導入することで、非常に深いネットワークでも効果的に学習を行うことができる。

RN50 および RN50x64

RN50 は、ResNet-50 をベースとしたモデルであり、50 層の深層ニューラルネットワークを用いて画像をエンコードする。RN50x64 は、ResNet-50 を 64 倍に拡張したモデルであり、より多くの特徴を捉えることができ、より高精度な画像認識が可能となる。これらのモデルは、CLIP の画像エンコーダとして機能し、テキストエンコーダと組み合わせることで、画像とテキストの関連性を高精度で捉えることができる。

主成分分析 (Principal Component Analysis, PCA)

主成分分析は、高次元データを低次元に変換するための統計手法である。データの分散を最大化する方向に新しい軸（主成分）を見つけ、その軸にデータを投影することで次元削減を行う。これにより、データの重要な特徴を保持しつつ、次元を削減することができる。各主成分の寄与度は、その主成分がデータの分散をどれだけ説明しているかを示す指標である。寄与度が高い主成分ほど、データの重要な情報を多く含んでいることを意味する。通常、最初の数個の主成分がデータの大部分の分散を説明するため、次元削減後のデータ解析において重要な役割を果たす。

K-means クラスタリング

K-means クラスタリングは、データを K 個のクラスに分割するための非階層型クラスタリング手法である。各クラスは、クラスを中心（重心）からの距離が最小になるようにデータポイントを割り当てることで形成される。K-means クラスタリングは、データのパターンを見つけるために広く使用される。

コサイン類似度

コサイン類似度は、2 つのベクトル間の類似度を測るための指標である。ベクトル間の角度のコサイン値を計算することで、ベクトルの方向がどれだけ似ているかを評価する。コサイン類似度は、0 から 1 の範囲で値を取り、1 に近いほどベクトルが類似していることを示す。

シード (Seed)

シードは、乱数生成の初期値を設定するための値である。シードを固定することで、実験の再現性を確保することができる。

ゼロショット学習 (Zero-Shot Learning)

ゼロショット学習は、トレーニング時に見たことのないクラスのデータを分類する能力を持つ機械学習手法である。ゼロショット学習では、事前に学習した知識を利用して、新しいクラスのデータを認識する。これにより、少ないデータで新しいクラスを分類することが可能となる。

平均二乗誤差 (Mean Squared Error, MSE)

平均二乗誤差は、予測値と実際の値の差の二乗の平均を計算する指標である。MSE は以下の式で計算される：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

ここで、 y_i は実際の値、 $\hat{y}_i$ は予測値、 n はデータポイントの数である。MSE は、モデルの予測精度を評価するために広く使用され、値が小さいほど予測精度が高いことを示す。

混合行列

混合行列 (Confusion Matrix) は, 分類モデルの性能を評価するための表であり, 以下の 4 つの要素から構成される.

- True Positives (TP): 正しく物体が存在すると判定された画像の数
- False Positives (FP): 誤って物体が存在すると判定された画像の数
- True Negatives (TN): 正しく物体が存在しないと判定された画像の数
- False Negatives (FN): 誤って物体が存在しないと判定された画像の数

F2-score

F2-score は, 精度 (Precision) と再現率 (Recall) の調和平均を計算するための指標であり, 再現率をより重視する場合に使用される. F2 スコアは以下の式で計算される:

$$F2 = \frac{(1 + 2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}}$$

ここで, 精度 (Precision) と再現率 (Recall) は以下のように計算される.

- 精度 (Precision) : $\frac{TP}{TP+FP}$
- 再現率 (Recall) : $\frac{TP}{TP+FN}$

F2-score は 0 から 1 の範囲で値を取り, 1 に近いほどモデルの性能が高いことを示す. F2-score が高い場合, モデルは高い再現率を維持しつつ, 精度も高いことを意味する. 逆に, F2-score が低い場合, モデルは再現率または精度のいずれか, または両方が低いことを示す. F2-score は, 特に不均衡データセットにおいて, モデルの性能を評価するために有用である.

正規化

正規化は, データのスケールを統一するための前処理手法であり, 機械学習モデルの学習を安定させるために使用される. OpenCV ライブラリでは, `cv2.normalize` 関数を使用して画像の正規化を行うことができる.

- **cv2.normalize:** `cv2.normalize` 関数は, 画像のピクセル値を指定された範囲にスケールングするために使用される. 最も一般的に使用される正規化方法は `cv2.NORM_MINMAX` であり, 以下の式で計算される.

$$\text{dst}(x, y) = \alpha + \frac{(\text{src}(x, y) - \min_{\text{src}})}{(\max_{\text{src}} - \min_{\text{src}})} \cdot (\beta - \alpha)$$

ここで, $\min_{\text{src}}$ と $\max_{\text{src}}$ は入力画像の最小値と最大値である.

正規化を行うことで, 異なるスケールの特徴量が同等に扱われるようになる.

膨張と収縮

膨張と収縮は、画像処理における形態学的操作であり、ノイズ除去や形状の強調に使用される。OpenCV ライブラリでは、`cv2.dilate` 関数と `cv2.erode` 関数を使用してこれらの操作を行うことができる。

- 膨張 (**Dilation**) : 膨張は、画像中の白い領域（前景）を拡大する操作である。膨張は、特定の構造要素（カーネル）を使用して行われ、前景のピクセルが周囲のピクセルに広がる。これにより、小さな穴や隙間が埋められる。

$$\text{dst}(x, y) = \max_{(i, j) \in \text{kernel}} \text{src}(x + i, y + j)$$

ここで、kernel は膨張に使用される構造要素である。

- 収縮 (**Erosion**) : 収縮は、画像中の白い領域（前景）を縮小する操作である。収縮は、特定の構造要素（カーネル）を使用して行われ、前景のピクセルが周囲のピクセルに侵食される。これにより、小さなノイズや細い線が除去される。

$$\text{dst}(x, y) = \min_{(i, j) \in \text{kernel}} \text{src}(x + i, y + j)$$

ここで、kernel は収縮に使用される構造要素である。

膨張と収縮を組み合わせることで、画像のノイズ除去や形状の強調が効果的に行える。例えば、膨張の後に収縮を行う「クロージング」操作や、収縮の後に膨張を行う「オープニング」操作がある。

2.2 検出方法

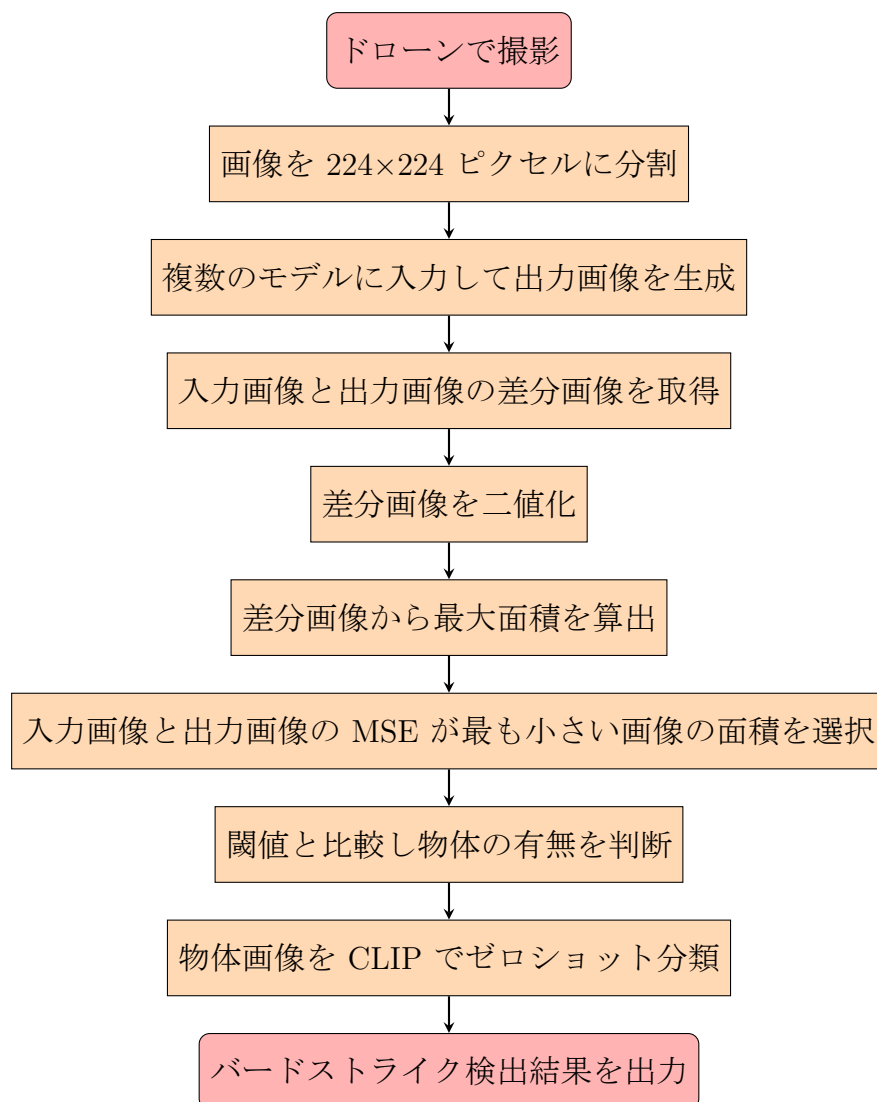


図 2.1: バードストライク検出システムの流れ

ドローン画像からコンピュータビジョンを用いてバードストライクを検出するシステムを構築した。システムの流れは図2.1の通りである。

始めに、ドローンで探索範囲を撮影し、画像（3000×4000 ピクセルが複数枚）を準備する。次に、それらの画像を 224×224 ピクセルに分割し、複数のモデルに入力することで出力画像を生成する。出力画像と入力画像の差分を取ることで、物体部分だけが浮き彫りになった差分画像を生成し、差分画像の二値化を行い最大面積を算出する。複数のモデルを使用しているため、モデル数分の最大面積が算出される。最も適したモデルによる最大面積を選択するために、入力画像と出力画像の MSE(平均二乗誤差)の値が最も小さいモデルの最大面積を選択する。最大面積と閾値を比較することで物体の有無を判定する。抽出できた物体画像に関しては、その画像が鳥の画像なのかを判定する必要があるが、現状では鳥の画像が少ないため、教師あり学習は困難である。そこで、ゼロショットで分類できる CLIP を導入した。

2.3 閾値に関して

閾値は, 物体の有無を判定するための重要なパラメータである. 昨年度は 1200 を閾値として使用していた. これは昨年度の test データに対して設定していた値であり, 今年度は新たに設定する必要がある.



図 2.2: ミズナギドリの死骸を撮影した画像

図2.2は, ミズナギドリと呼ばれる鳥の死骸をドローンで撮影した画像である. 経験上ではあるが, この鳥がバードストライクが起きる鳥の中間サイズである. この画像を基準に新たに閾値を設定する.

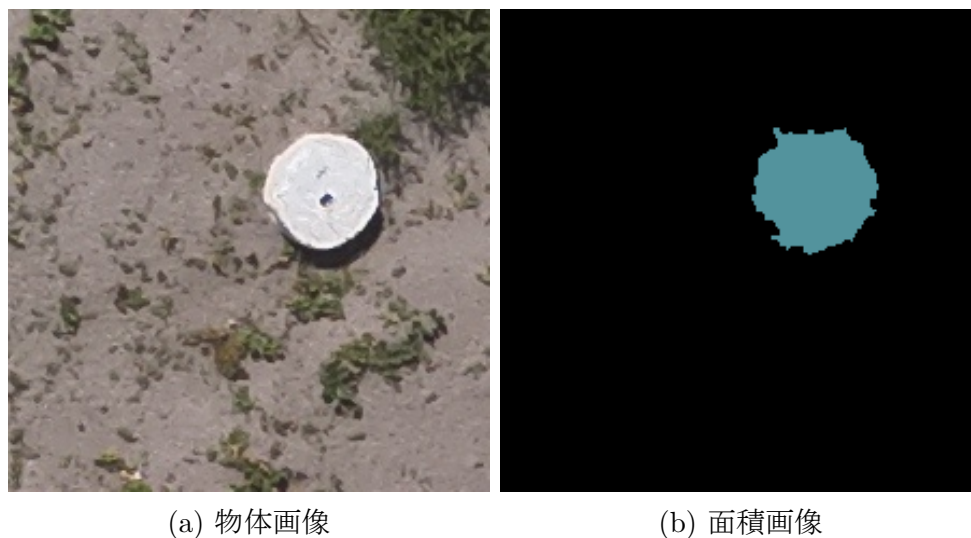


図 2.3: 物体画像と面積画像

図2.3aの面積を算出した画像が図2.3bである. この画像の面積は 2100 である. ミズナギドリが中間サイズであることを踏まえて, 閾値を 800 に設定する.

2.4 AE の改良

昨年度使用していた AE モデルと比較して、構造をシンプルにすることで性能向上を図った。以下に、以前のモデルと現在のモデルの変更点、及び変更による改善点について説明する。

モデル用語の説明

- **Conv2d** (入力, 出力): 2 次元畳み込み層. 入力チャンネル数と出力チャンネル数を指定し, 画像の特徴を抽出するために使用される. 例えば, Conv2d (3, 16) は 3 チャンネルの入力画像を 16 チャンネルの出力に変換する.
- **BatchNorm2d**: バッチ正規化層. ミニバッチごとにデータを正規化し, 学習の安定性と速度を向上させる. 内部共変量シフトを減少させる効果がある.
- **ReLU**: Rectified Linear Unit (ReLU) 活性化関数. 入力が 0 以下の場合は 0 を出力し, 0 より大きい場合はそのまま出力する. 非線形性を導入し, モデルの表現力を高める.
- **MaxPool2d**: 最大プーリング層. 指定されたウィンドウサイズ内で最大値を抽出し, 空間的なサイズを縮小する. 特徴マップのダウンサンプリングに使用される.
- **ConvTranspose2d** (入力, 出力): 2 次元逆畳み込み層. 畳み込み層の逆操作を行い, 特徴マップの空間的なサイズを拡大する. アップサンプリングに使用される.

昨年度の AE モデル

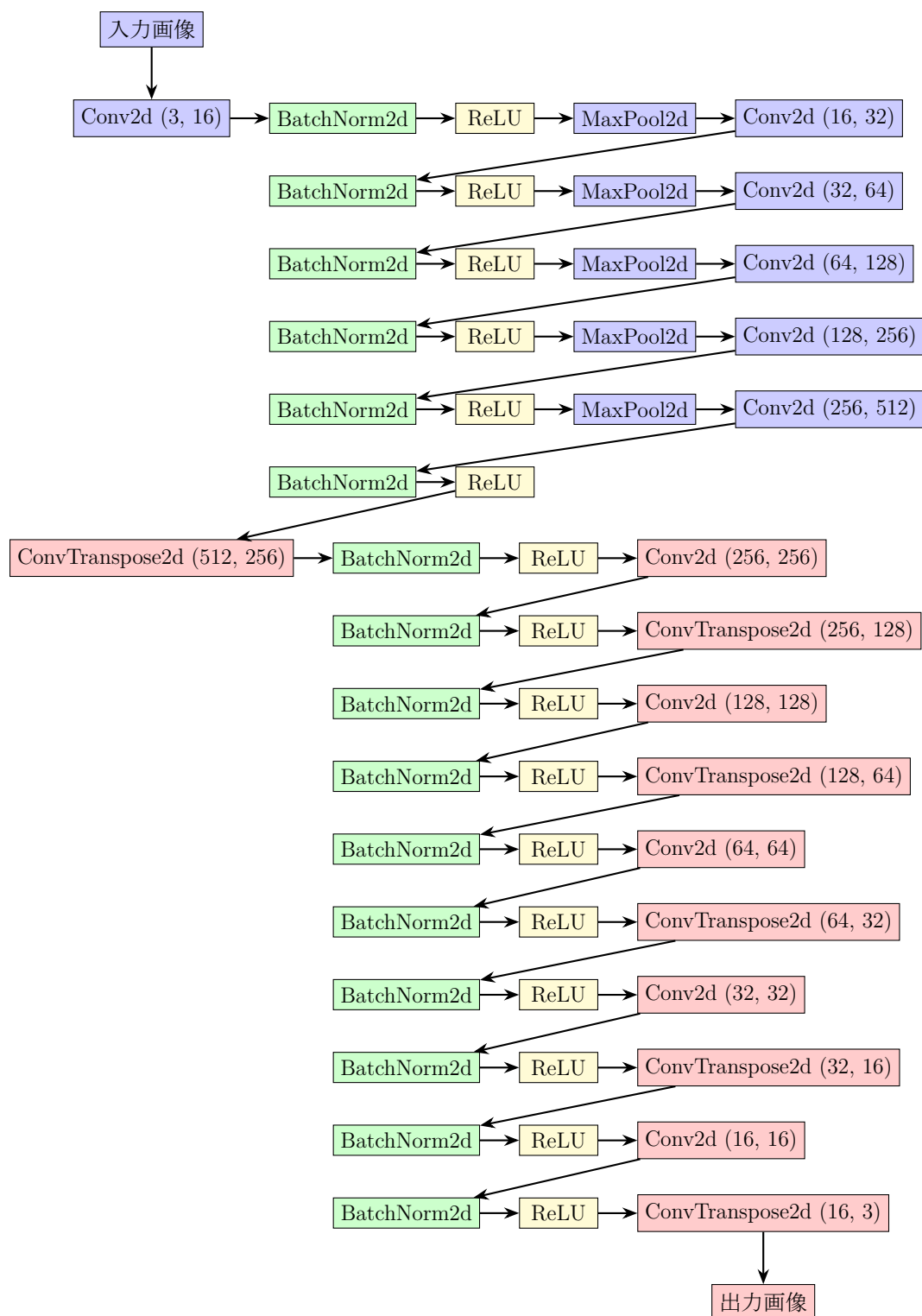


図 2.4: AE の構造

図2.4は、昨年度の AE モデルの構造を示している。

今年度の AE モデル

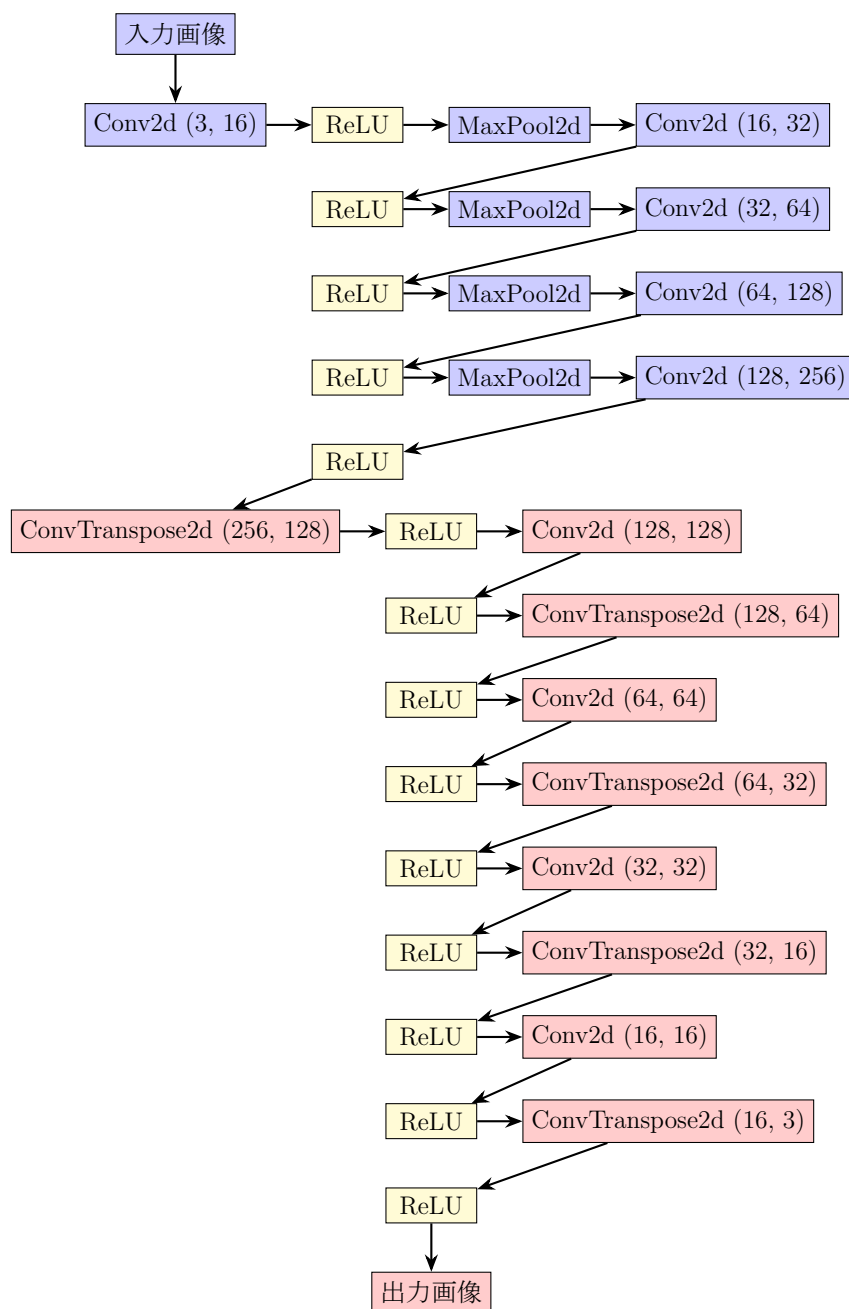


図 2.5: 今年度の AE の構造

図2.5は, 今年度の AE モデルの構造を示しており, 昨年度のモデルと比較して構造がシンプルになっている.

変更点

- 畳み込み層と逆畳み込み層の数を減らし, モデルの深さを浅くした.
- バッチ正規化層を削除し, 計算コストを削減した.
- ReLU 活性化関数の使用を最小限に抑えた.

改良の効果

これらの変更により、以下のような改善点が見られた。

- 過学習の防止: モデルのパラメータ数を減らすことで、過学習のリスクが低減された。これにより、テストデータに対する汎化性能が向上し、より安定した結果が得られるようになった。
- モデルの解釈性の向上: 構造をシンプルにすることで、モデルの挙動がより理解しやすくなった。これにより、モデルの改善点や問題点を特定しやすくなり、さらなる改良が容易になった。

以上のように、今年度の AE モデルは、昨年度のモデルと比較して構造がシンプルになり、過学習の防止、モデルの解釈性の向上といった改善点が見られた。

2.5 AE モデルのアップデートの重要性

AE で構築したモデルをアップデートすることは、モデルの性能向上や新しいデータへの適応において非常に重要である。以下に、モデルをアップデートせずに使い続けることと、アップデートして使うことの差について説明する。

モデルをアップデートせずに使い続けると、以下のような問題が発生する可能性がある。

- 性能の劣化: 新しいデータや環境に適応できず、モデルの性能が徐々に劣化する可能性がある。例えば、[2] では、モデルの性能がデータの変化に対して敏感であることが示されている。
- 過学習: 古いデータに対して過学習してしまい、新しいデータに対して一般化能力が低下する可能性がある。過学習は、モデルの汎化性能を低下させる主要な要因の一つだ [3]。

一方、モデルを定期的にアップデートすることで、以下のような利点がある。

- 性能向上: 新しいデータや技術を取り入れることで、モデルの性能を向上させることができる。例えば、[4] では、ResNet のような新しいアーキテクチャを導入することで、画像認識の精度が大幅に向上したことが示されている。
- 適応能力の向上: モデルが新しいデータや環境に適応できるようになり、汎化性能が向上する。これは、モデルがより広範なデータセットに対して有効であることを意味する [5]。

2.6 モデルのアップデート方法

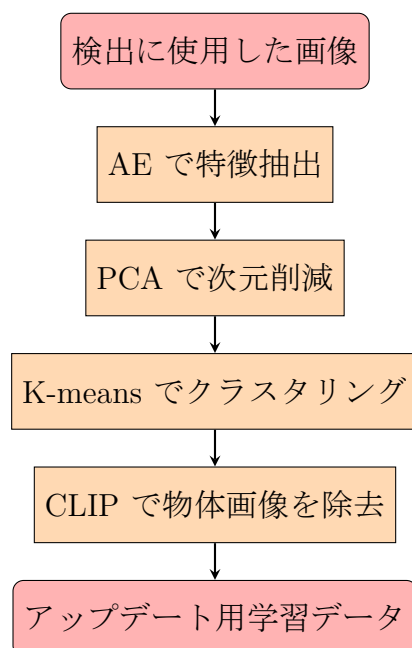


図 2.6: モデルのアップデート方法の流れ

検出に使用した画像を用いてモデルのアップデートを行う。実用化のために自動アップデートを行いたい。自動アップデートには学習データを自動で構築する必要がある。以下の二つのステップで構築できることが確認できた。

始めに、全体の画像を背景ごとにクラスタリングする。これは AE で特徴抽出を行い、特徴量を PCA（主成分分析）で次元削減し、K-means クラスタリングを用いることで可能であることが確認できた。

次に、画像の中から物体画像を取り除く。これは CLIP を使用することで効果的に行うことができる。CLIP を用いることで、画像内の物体をある程度の精度で検出し、物体が存在しない画像を抽出することができる。これらのステップに関しては、3 章で詳しく説明する。

構築したアップデート用学習データを用いて、AE モデルをファインチューニングすることで、モデルのアップデートを行う。このとき、層ごとにパラメータの調整は行わずに、均一なパラメータで学習を行う。

3 実験

本章では、従来の方法で物体検出を試みたうえでモデルアップデートの有用性について論じる。なお、使用した環境や画像、モデル、プログラミングに関しては 6 章で詳細に説明を行っているため、そちらを参照して欲しい。また、ファイル名で使われている 8 桁の数字は日付を表していて、実行した日付によって異なる事に注意する。

3.1 物体検出の試み

従来の方法で物体検出を試みる。結果は以下の通りである。

使用プログラム

- **2023_main.py**

使用画像

- **imgs/test_img**
- **imgs/test_objects**

使用モデル

- **models/2023models/AEmodel_dark_green20230927.pth**
- **models/2023models/AEmodel_light_green20230927.pth**
- **models/2023models/AEmodel_white20230927.pth**
- **models/2023models/AEmodel_paved_ground20230927.pth**

出力

物体画像の判定が行われ、F2-score が計算される。

出力先

- **imgs/AE_2023_result**: 物体と判定された画像が保存される。

F2-score の計算

システムが物体画像と判定した画像の枚数は 459 枚で、その中に入っていた物体画像の枚数は 12 枚である。imgs/test_img の画像を分割した画像の枚数が 884 枚で、その中に入っていた物体画像の枚数が 13 枚である。混合行列の要素は以下の通りである。

- 混合行列の要素:
 - True Positives (TP): 12 枚
 - False Positives (FP): 447 枚
 - True Negatives (TN): 424 枚
 - False Negatives (FN): 1 枚
- 精度 (Precision) と再現率 (Recall) を計算する。

$$- \text{Precision} = \frac{TP}{TP+FP} = \frac{12}{12+447} \approx 0.026$$

$$- \text{Recall} = \frac{TP}{TP+FN} = \frac{12}{12+1} \approx 0.923$$

- F2 スコアを計算する.

$$- F2 = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.026 \cdot 0.923)}{4 \cdot 0.026 + 0.923} \approx 0.117$$

従来のモデルでは物体検出の精度が十分でないことが確認できた. 特に FN が 1 以上だと, 見落としてしまう可能性があるため確実に避けたい. そのためにもモデルのアップデートを行うことが必要である.

3.2 モデルアップデートの有用性

モデルアップデートの有用性を検証するために, 自動でモデルのアップデートを行うシステムを構築する.

3.3 背景画像の分類

初めに, AE の性能を最大限発揮するためにモデルの対象ごとに画像を分類することが必要である. これから背景ごとに画像を分類する方法を説明する.

通常, AE モデルを構築するときは画像の特徴の圧縮と再構築を行い入力画像と出力画像の一致率 (MSE) を高めようとする性質を利用するために学習画像は類似した画像を使用する. しかし, 今回は 4 種類の異なった画像を均一な枚数学習させることによって, それぞれの特徴を学習したモデルを構築する. この AE モデルで対象とする画像に対して一番圧縮された箇所のベクトルを抽出し, 主成分分析を行うことで次元削減を行う. その後, K-means クラスタリングを行うことで背景ごとに画像を分類することができる.

検証

検証を行う. 結果は以下の通りである.

使用プログラム

- AE_train.py

使用画像

- imgs/old_all_img/train_img_6048/6048/A_train_img

出力

AE モデルが構築される.

出力モデル名

- models/6048_AEdeepmodel_20241224_A_train_img.pth

使用プログラム

- AE_pca_kmeans.ipynb

使用画像

- `imgs/make_pca_img1000`

使用モデル

- `models/6048_AEdeepmodel_20241224_A_train_img.pth`

出力

入力画像の圧縮, ベクトルの主成分分析, 寄与度の確認, 二次元へのプロット, クラスタ数 10 での K-means クラスタリングを行った.

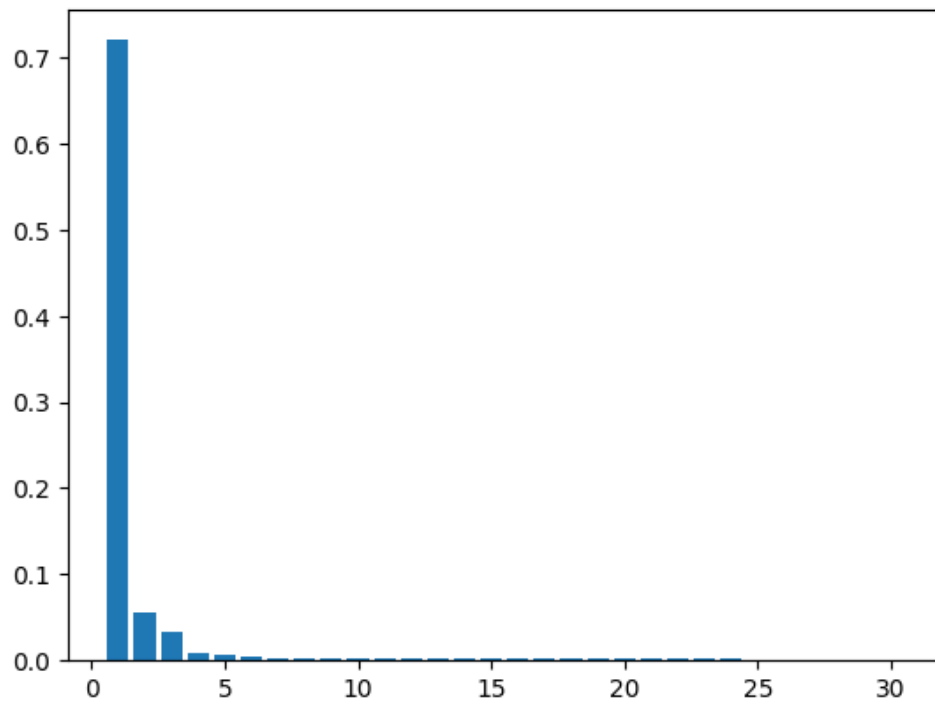


図 3.1: 主成分分析の寄与度

図3.1は, 主成分分析の寄与度を示すグラフであり, 各主成分がデータの分散にどの程度寄与しているかを確認することができる. 結果より, ほとんどの寄与度が第一主成分と第二主成分で占められていることが確認できる. したがって, 二次元にプロットすることで十分にデータの特徴を表現できると考えられる.

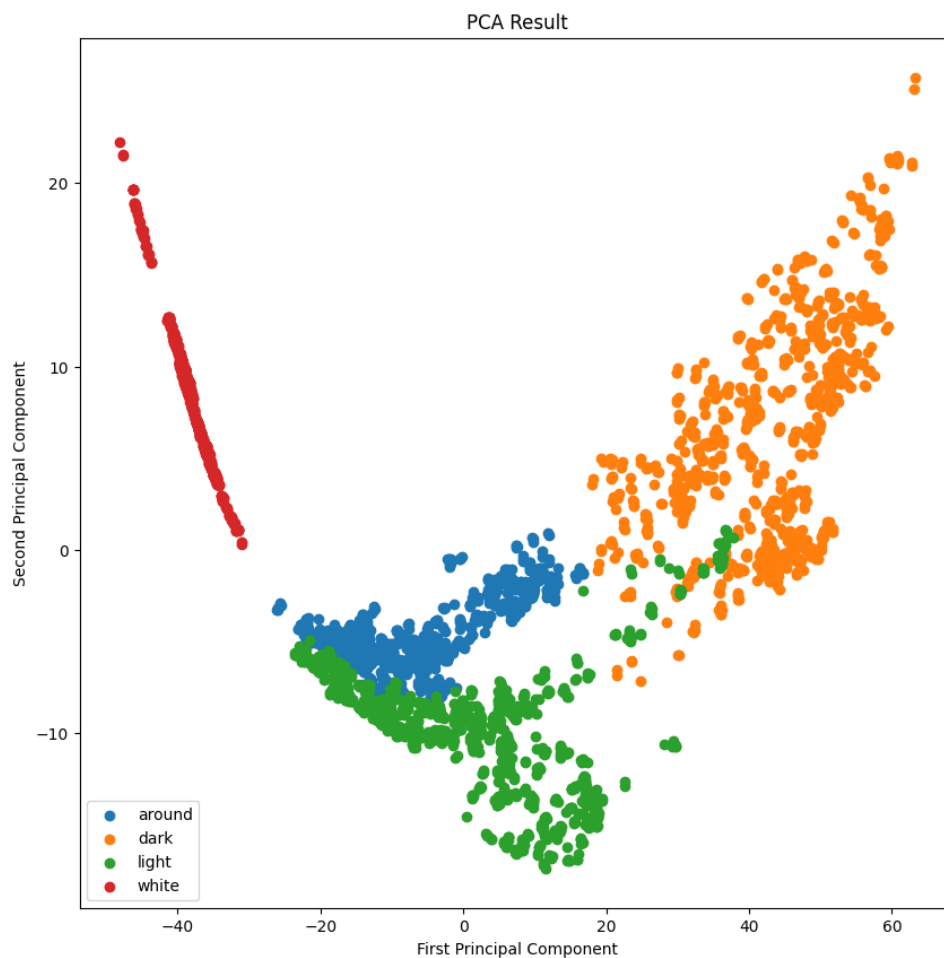


図 3.2: 主成分分析による二次元プロット

図3.2は、主成分分析によって次元削減されたデータの二次元プロットである。このプロットにより、データの分布やクラスタリングの傾向を視覚的に確認することができる。結果より、around と light が近く、その他は離れていることが確認できる。このことから、around と light は似た特徴を持っていることが示唆されるため、合わせて1種類の背景とし light と扱うことにする。そのため、以降は背景画像は3種類とする。

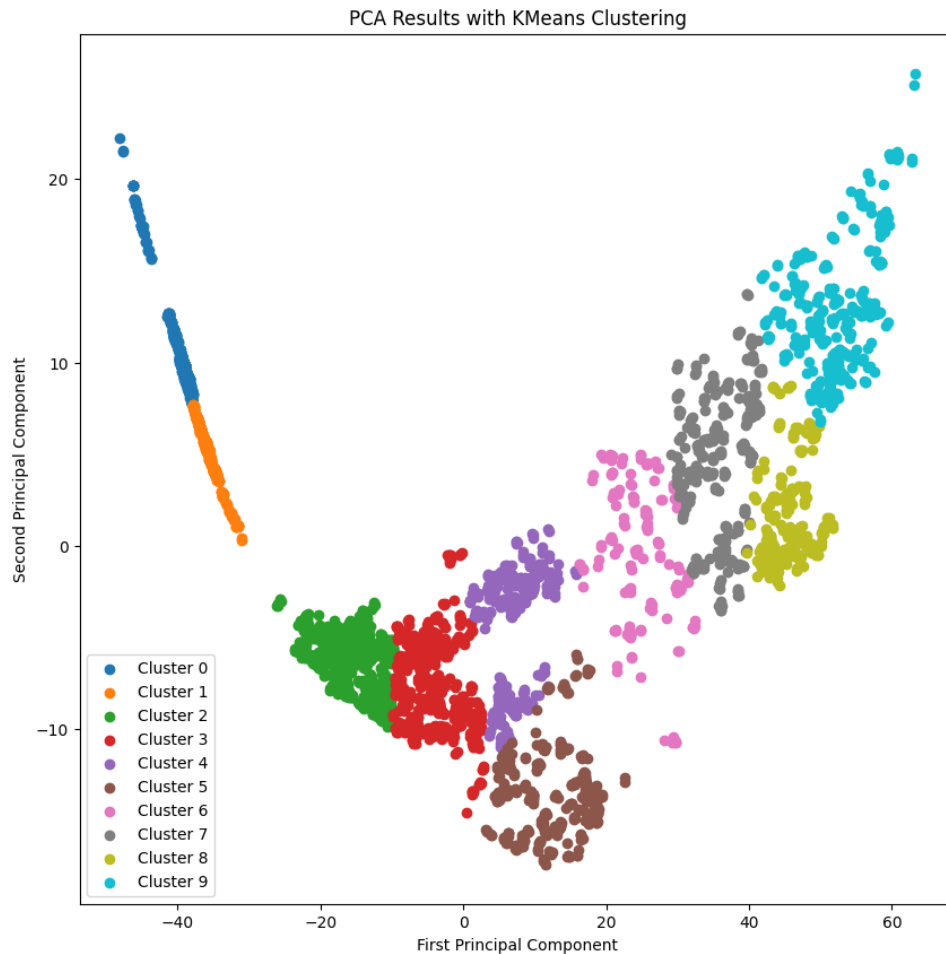


図 3.3: K-means クラスタリングの結果

図3.3は、主成分分析による次元削減結果に対してクラスター数 10 の K-means クラスタリングの結果を示すプロットである。このプロットにより、データがどのようにクラスターに分割されたかを視覚的に確認することができる。横軸の値が小さいクラスターから順に数字を 0 から 9 まで指定することで結果が固定される。結果より、0 と 1 は white, 4 と 5 は light, 8 と 9 は dark とすることで曖昧な境界部分は除かれ、画像を背景ごとに分類することができる。

実証

実証を行う。結果は以下の通りである。

使用プログラム

- `AE_make_dataset.ipynb`

使用画像

- `imgs/for_update_img`

使用モデル

- `models/6048_AEdeepmodel_20241224_A_train_img.pth`

出力

入力画像の圧縮, ベクトルの主成分分析, 二次元へのプロット, クラスター数 10 での K-means クラスタリングを行い, 背景ごとに分類した. この時, 主成分分析に使用したモデルは `AE_score_check_labels.ipynb` で使用したモデルと同じことに注意する.

出力先

- `imgs/update_img`
 - `dark`
 - `light`
 - `white`

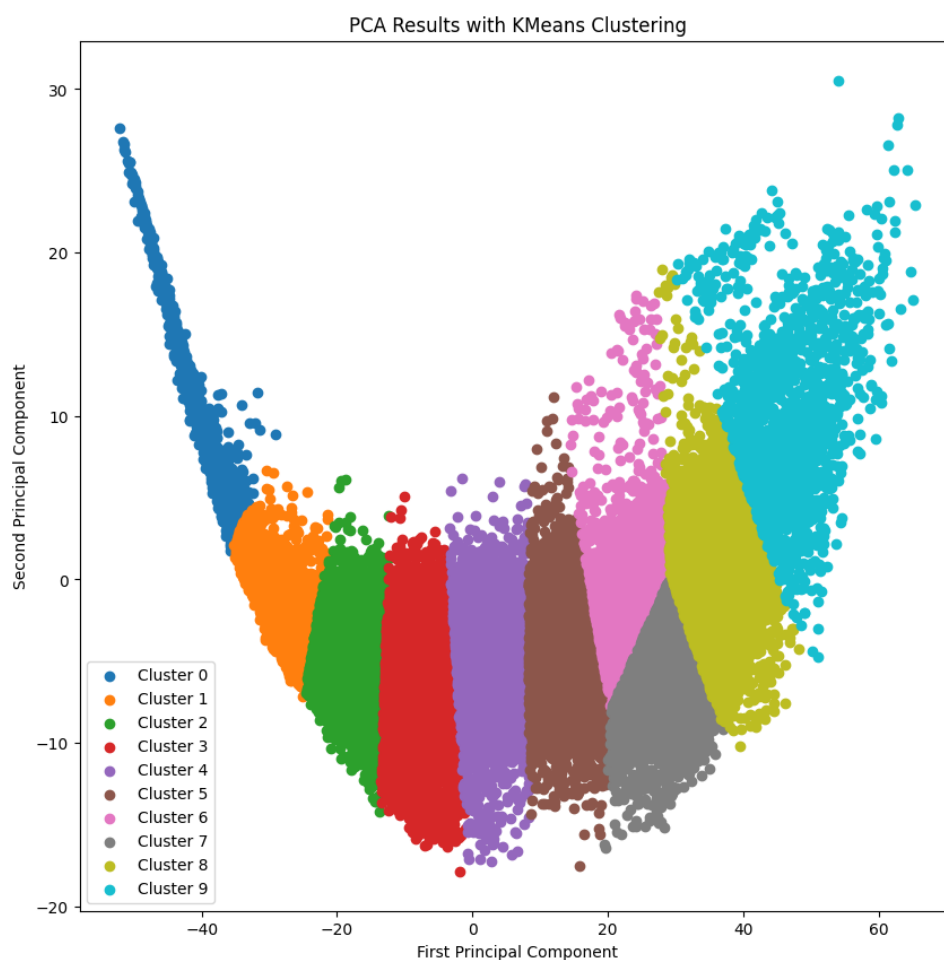


図 3.4: K-means クラスタリングの結果

3.4 物体画像の抽出

次に, 背景ごとに分類された画像から物体画像を抽出する. これは, CLIP モデルを使用して画像内の物体を検出し, 物体が存在する画像を抽出することで行う. また,

ここで抽出したい物体画像は最終的に検出したい物体よりも小さい物体が含まれていることに注意する。これは、最終的に検出したい物体画像は閾値を越える大きさの物体を想定しているが、今回抽出したい物体画像は AE の性能を十分に発揮するために大きさは問わないためである。

検証と実証

検証と実証を行う。結果は以下の通りである。

使用プログラム

- **AE_clip.ipynb**

使用画像

- **imgs/update_img/**
 - **dark**
 - **light**
 - **white**
- **imgs/update_img_remove/**

使用モデル

- **RN50x64(High Model)**
- **RN50(Low Model)**

使用した言葉

本研究で使用した言葉は以下の通りである。これらの言葉は CLIP モデルに入力するために GoogleTranslator で英語に翻訳して使用された。

- **NO_A:** 上空から緑色や黄緑色の草が生い茂った箇所を見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_B:** 上空から緑色や黄緑色の草が砂浜の上から生えているのを見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_C:** 上空から緑色の葉が生えた木と隙間に砂浜が見られる箇所画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_D:** 上空からクリーム色の砂浜の上に少し草が生えている箇所を見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_E:** 上空からクリーム色の砂浜を見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない

- **YES_A:** 上空から緑色や黄緑色の草が生い茂った箇所を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_B:** 上空から緑色の葉が生えた木を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_C:** 上空から緑色や黄緑色の草が砂浜の上から生えているのを見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_D:** 上空から緑色の葉が生えた木と隙間に砂浜が見られる箇所で見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_E:** 上空からクリーム色の砂浜の上に少し草が生えている箇所を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_F:** 上空からクリーム色の砂浜を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある

出力

プログラムとモデルと画像を用いて物体画像を抽出し、学習データを構築した。また、物体画像を抽出したデータを用意することで2つのモデルにおける精度比較も行った。100%物体画像を取り除くことはできなかったため、手動で物体画像を完全に取り除いた画像も保存し、それぞれの画像で学習を行いモデルの精度比較に使用する。学習データを構築する際は、ランダムに上下反転、左右反転、回転を行い、データ拡張を行った。

出力先

- **imgs/clip_test:** テストデータ
- **imgs/clip_test_result:** テストデータの出力結果
- **imgs/update_train_imgs_default/train_img_6048:** imgs/update_img/に対して物体抽出を行い、データ拡張をした学習用データ
- **imgs/update_train_imgs_remove/train_img_6048:** imgs/update_img/に対して物体抽出を行い、imgs/update_img_remove/を使用して完全に物体画像を取り除きデータ拡張をした学習用データ

テストデータの結果

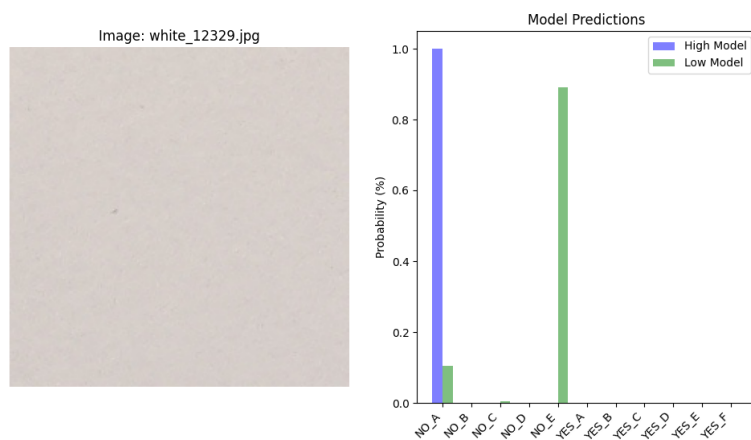


図 3.5: テスト画像 1 の予測結果

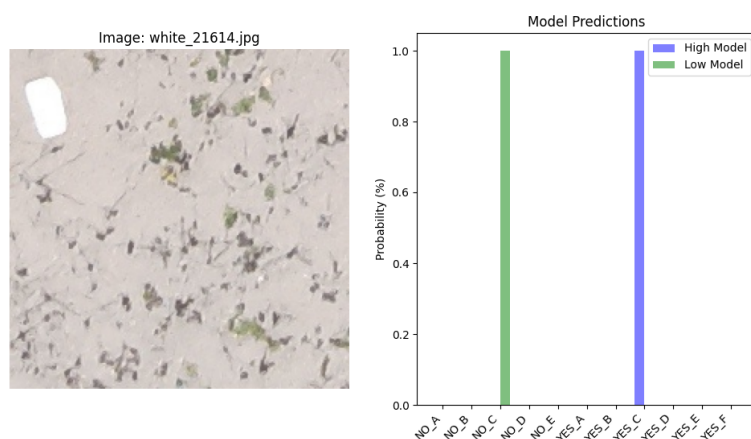


図 3.6: テスト画像 2 の予測結果

精度を向上させるためにテストデータを用いて、入力する言葉の微調整を行った。図3.5は物体が存在しない画像を入力した結果であり、図3.6は物体が存在する画像を入力した結果である。棒グラフは、入力画像に対して各モデルの予測値の割合を示している。

抽出結果

以下に、モデルごとの抽出結果を示す。

表 3.1: RN50x64 モデルと RN50 モデルの物体を含んでいる割合の比較

カテゴリ	RN50x64	RN50
dark	1.26%	4.64%
light	11.76%	15.33%
white	20.51%	29.98%

RN50x64 モデルと RN50 モデルの比較

RN50x64 モデルと RN50 モデルの結果を比較すると、RN50x64 モデルの方が物体を含んでいる割合が全てのカテゴリで低いことが確認できる。結果から、RN50x64 モデルを使用して物体画像を抽出することが適していたため以降は RN50x64 モデルを使用した学習データを用いて実験を進める。なお、上記の表からも確認できるように、完全に物体を取り除くことは不可能だったため、以降は若干物体の画像が入った画像でモデルのアップデートを行った場合と全く物体が入っていない画像でモデルのアップデートを行った場合の精度の比較も行う。

3.5 モデルのアップデート

構築した学習データでモデルのアップデートを行う。モデルのアップデートは、古い画像を学習させたモデルを新しい画像でアップデートすることで行う。また、背景画像は 4 分類ではなく 3 分類の方が適していることが確認できたことや、AE モデルの構造を改善したためアップデートをするモデルも作り直す。

旧モデルの再学習

旧モデルの再学習を行う。結果は以下の通りである。

使用プログラム

- AE_train.py

使用画像

- imgs/old_train_img3/train_img_6048/6048

出力

AE モデルが構築される。

出力モデル名

- models/old_models6048/20241225_new_dark.pth
- models/old_models6048/20241225_new_light.pth
- models/old_models6048/20241225_new_white.pth

モデルのアップデート

モデルのアップデートを行う。結果は以下の通りである。

使用プログラム

- AE_trainf.py

使用画像

- imgs/update_train_imgs_default/train_img_6048

使用モデル名

- models/old_models6048/20241225_new_dark.pth
- models/old_models6048/20241225_new_light.pth
- models/old_models6048/20241225_new_white.pth

出力

AE モデルのアップデートを行った。また、以降では出力されたモデルをまとめて **dmodel** と呼ぶことにする。

出力モデル名

- models/fine_model_paths/6048_fineAEdeepmodel_20250115_ddark.pth
- models/fine_model_paths/6048_fineAEdeepmodel_20250115_dlight.pth
- models/fine_model_paths/6048_fineAEdeepmodel_20250115_dwhite.pth

使用プログラム

- AE_trainf.py

使用画像

- imgs/update_train_imgs_remove/train_img_6048

使用モデル名

- models/old_models6048/20241225_new_dark.pth
- models/old_models6048/20241225_new_light.pth
- models/old_models6048/20241225_new_white.pth

出力

AE モデルのアップデートを行った。また、以降では出力されたモデルをまとめて **rmodel** と呼ぶことにする。

出力モデル名

- models/fine_model_paths/6048_fineAEdeepmodel_20250115_rdark.pth
- models/fine_model_paths/6048_fineAEdeepmodel_20250115_rlight.pth
- models/fine_model_paths/6048_fineAEdeepmodel_20250115_rwhite.pth

3.6 物体検出の試み

アップデートした dmodel と rmodel を使って、再び物体検出を試みた。結果は以下の通りである。

使用プログラム

- 2024_main.py

使用画像

- `imgs/test_img`
- `imgs/test_objects`

使用モデル

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_ddark.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_dlight.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_dwhite.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rdark.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rlight.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rwhite.pth`

出力

物体画像の判定が行われ, F2-score が計算される.

出力先

- `imgs/AE_2024_dresult`: dmodel を使用して物体と判定された画像が保存される.
- `imgs/AE_2024_rresult`: rmodel を使用して物体と判定された画像が保存される.

F2-score の計算

以下に, dmodel と rmodel の F2-score を示す.

dmodel の F2-score

システムが物体だと判定した画像の枚数は 348 枚で, その中に入っていた物体画像の枚数は 12 枚である. `imgs/test_img` の画像を分割した画像の枚数が 884 枚で, その中に入っていた物体画像の枚数が 13 枚である. 混合行列の要素は以下の通りである.

- 混合行列の要素:
 - True Positives (TP): 12 枚
 - False Positives (FP): 336 枚
 - True Negatives (TN): 535 枚
 - False Negatives (FN): 1 枚
- 精度 (Precision) と再現率 (Recall) を計算する.
 - $\text{Precision} = \frac{TP}{TP+FP} = \frac{12}{12+336} \approx 0.034$
 - $\text{Recall} = \frac{TP}{TP+FN} = \frac{12}{12+1} \approx 0.923$
- F2-score を計算する.
 - $$F2 = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.034 \cdot 0.923)}{4 \cdot 0.034 + 0.923} \approx 0.15$$

rmodel の F2-score

システムが物体だと判定した画像の枚数は 345 枚で, その中に入っていた物体画像の枚数は 11 枚である. imgs/test_img の画像を分割した画像の枚数が 884 枚で, その中に入っていた物体画像の枚数が 13 枚である. 混合行列の要素は以下の通りである.

- 混合行列の要素:
 - True Positives (TP): 11 枚
 - False Positives (FP): 334 枚
 - True Negatives (TN): 537 枚
 - False Negatives (FN): 2 枚
- 精度 (Precision) と再現率 (Recall) を計算する.

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{11}{11+334} \approx 0.032$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{11}{11+2} \approx 0.846$$

- F2-score を計算する.

$$\text{F2} = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.032 \cdot 0.846)}{4 \cdot 0.032 + 0.846} \approx 0.139$$

アップデート前の F2-score が 1.17 であったため, 若干だが性能が向上していることが確認できる. しかし前提として F2-score よりも FN が 1 以上であることが問題であるため, 物体検出システムを共通で検出不可だった画像を使用して問題点を確認する. また, dmodel と rmodel の両モデルで確認を行う.

3.7 物体検出システムの確認



図 3.7: 物体検出に失敗した画像

図3.7は, 物体検出に共通で失敗した画像である. この画像を用いて, 物体検出システムの問題点を特定する. ここで着目したいのは, どの段階で物体を見失っているかである.

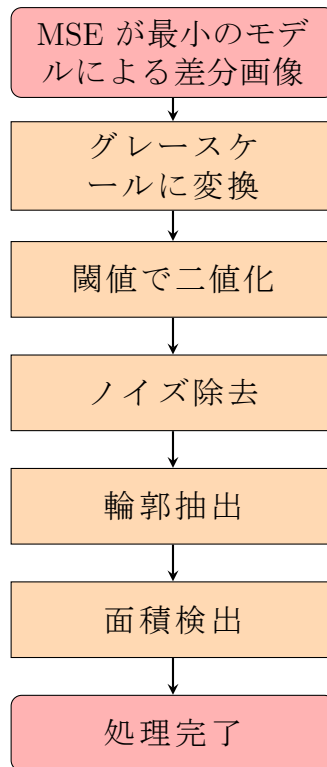


図 3.8: 差分画像の処理フロー

物体を検出するために、図3.8の処理を行った。1 つずつ処理を確認することで、物体を見失っている原因を特定する。結果は以下の通りである。

使用プログラム

- `AE_inference.ipynb`

使用画像

- `imgs/test_img`

使用モデル（実行するとモデルが移動するため、移動後のパス）

- `models/dmodels/6048_fineAEdeepmodel_20250115_ddark.pth`
- `models/dmodels/6048_fineAEdeepmodel_20250115_dlight.pth`
- `models/dmodels/6048_fineAEdeepmodel_20250115_dwhite.pth`
- `models/rmodels/6048_fineAEdeepmodel_20250115_rdark.pth`
- `models/rmodels/6048_fineAEdeepmodel_20250115_rlight.pth`
- `models/rmodels/6048_fineAEdeepmodel_20250115_rwhite.pth`

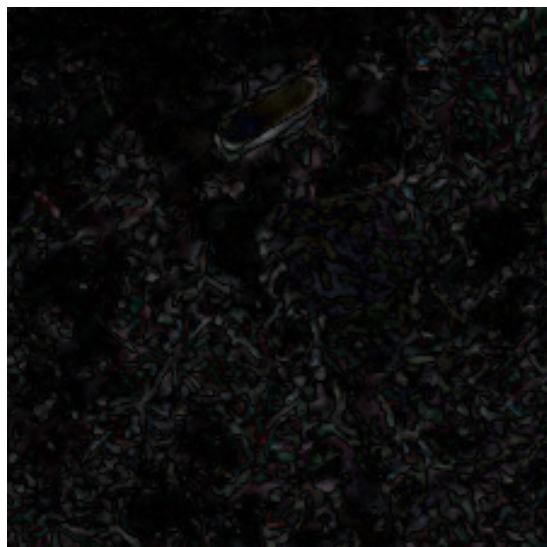
出力

このプログラムではコード内の引数に応じて、各処理の結果を出力する。

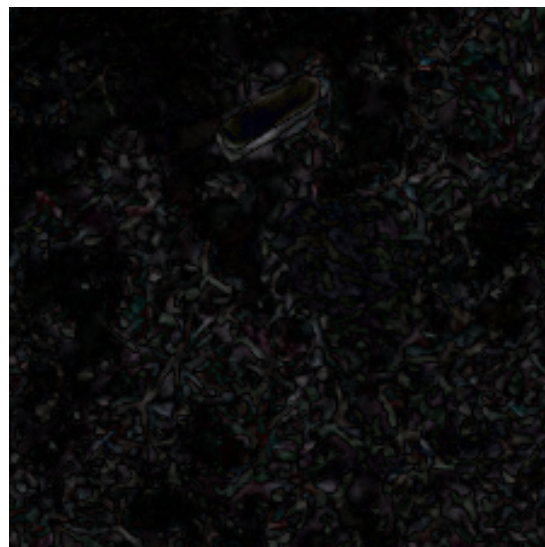
出力先

- `imgs/d_search_result`: dmodel を使用した検出過程の画像が保存される。
- `imgs/r_search_result`: rmodel を使用した検出過程の画像が保存される。

グレースケールに変換



(a) dmodel によるグレースケール画像

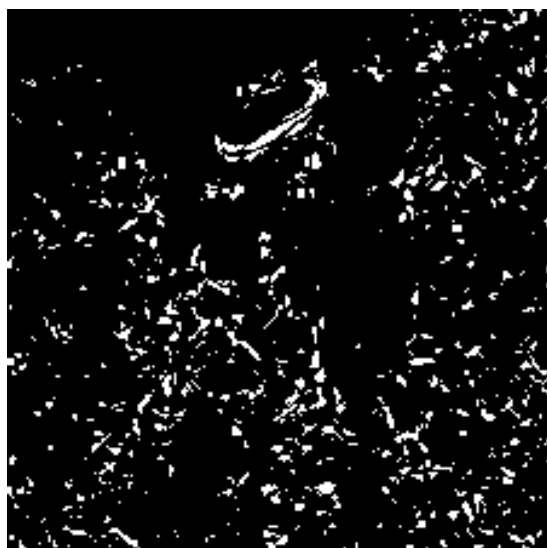


(b) rmodel によるグレースケール画像

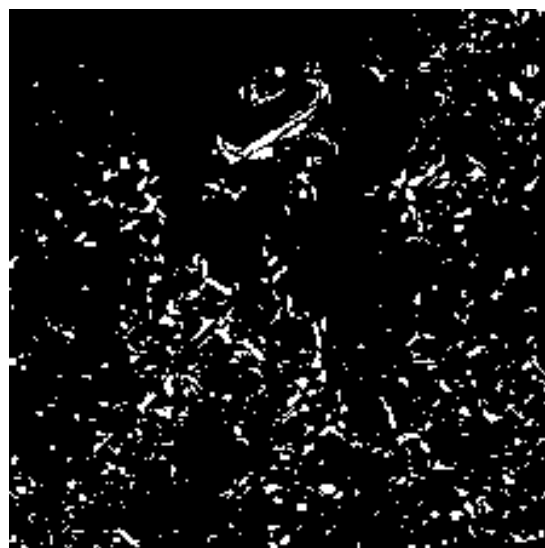
図 3.9: 物体検出に失敗した画像のグレースケール変換結果

図3.9は, 物体検出に失敗した画像を dmodel と rmodel でグレースケールに変換した結果を示す. 視覚的には, 物体が確認できている.

二値化



(a) dmodel による二値化画像

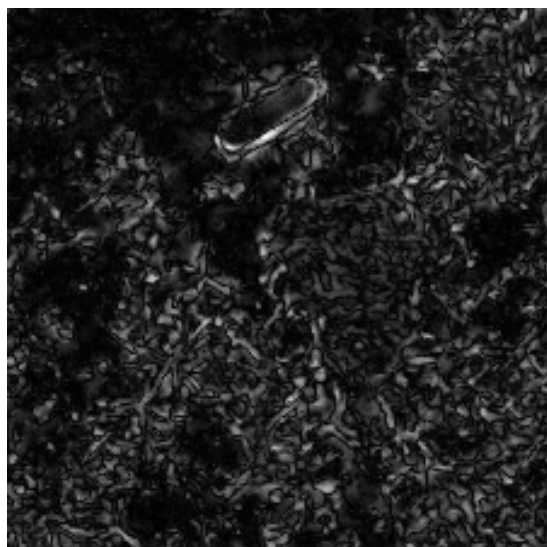


(b) rmodel による二値化画像

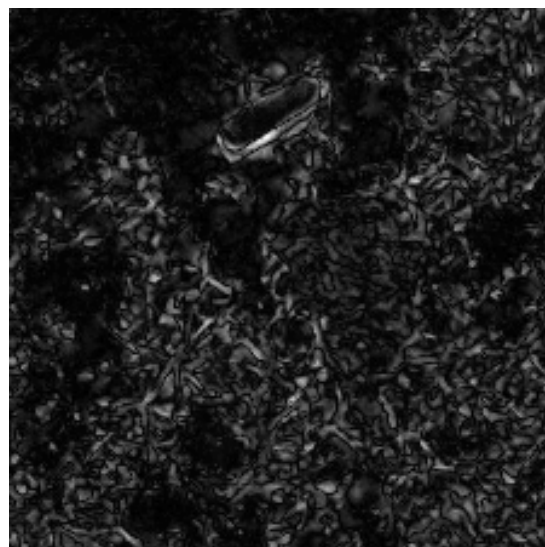
図 3.10: 物体検出に失敗した画像の二値化結果

図3.10は, 物体検出に失敗した画像を dmodel と rmodel で二値化した結果を示す. 二値化の閾値は 30 とした. 明らかに物体部分が途切れてしまっている. これを解決するために, 二値化を行う前に正規化を行い, 画像のコントラストを高める.

正規化後に二値化

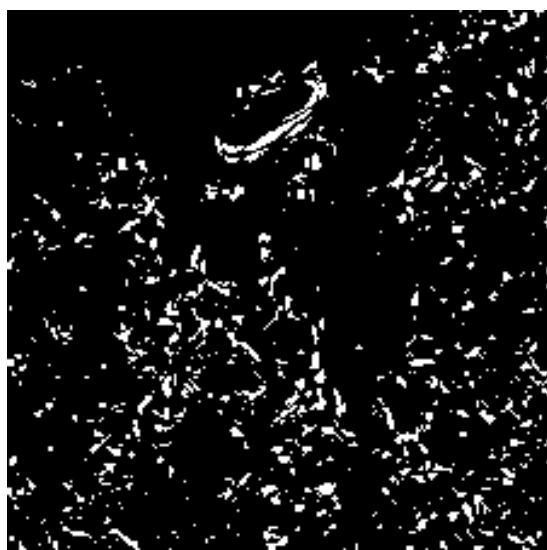


(a) dmodel による正規化画像

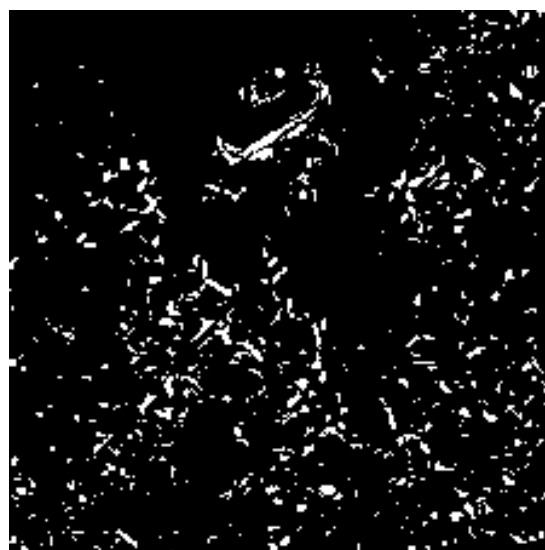


(b) rmodel による正規化画像

図 3.11: 物体検出に失敗した画像の正規化



(a) dmodel による二値化画像

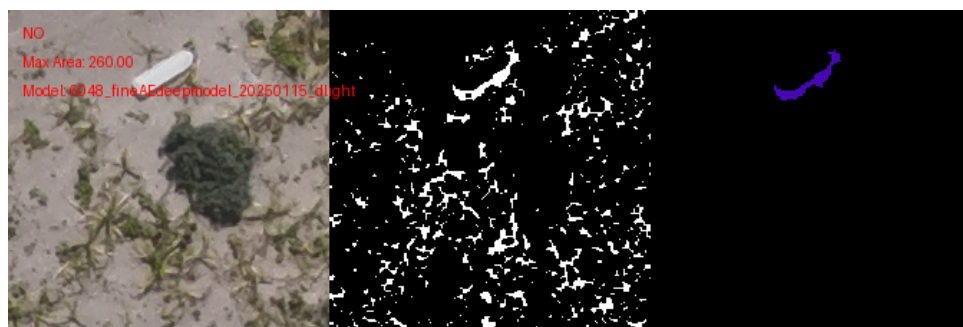


(b) rmodel による二値化画像

図 3.12: 物体検出に失敗した画像の正規化と二値化

図3.12は, 物体検出に失敗した画像を dmodel と rmodel で正規化と二値化の結果を示す. 閾値は 80 とした. 正規化を行うことで, 物体部分が途切れることが改善されることはなかった. そのため, 次の方法として二値化した画像に対してノイズ除去を行い, クロージングを行うことで物体部分が途切れることの解消を目指し, 物体が検出できているのかの確認を行う.

ノイズ除去及びクロージング

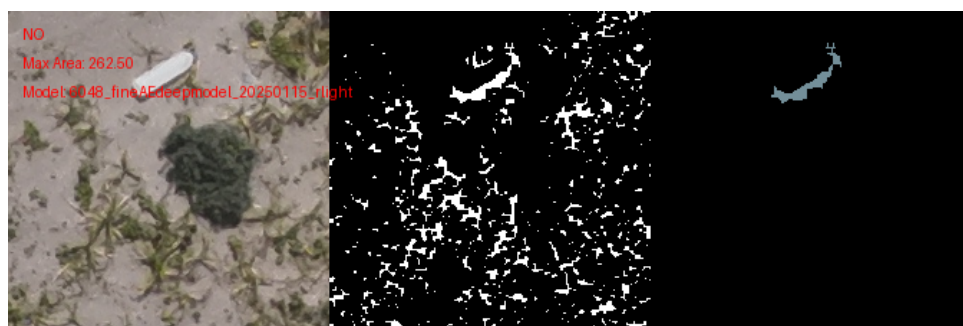


(a) dmodel によるノイズ除去画像

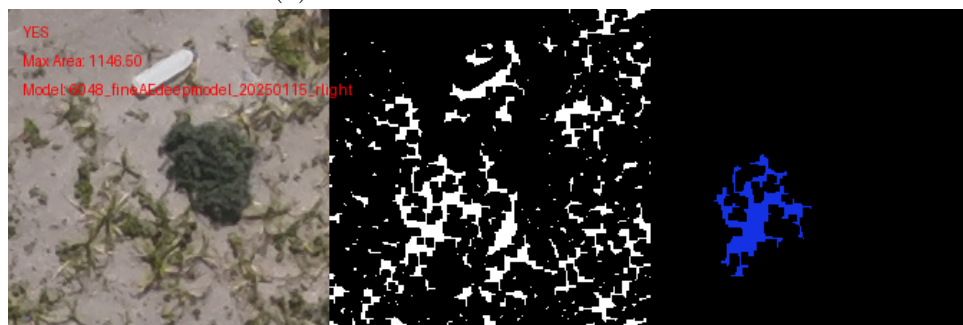


(b) dmodel によるノイズ除去及びクロージング画像

図 3.13: 正規化なし dmodel 検出結果

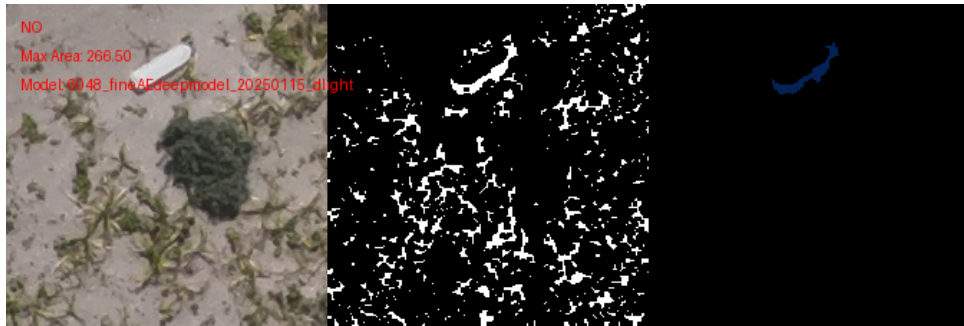


(a) rmodel によるノイズ除去画像

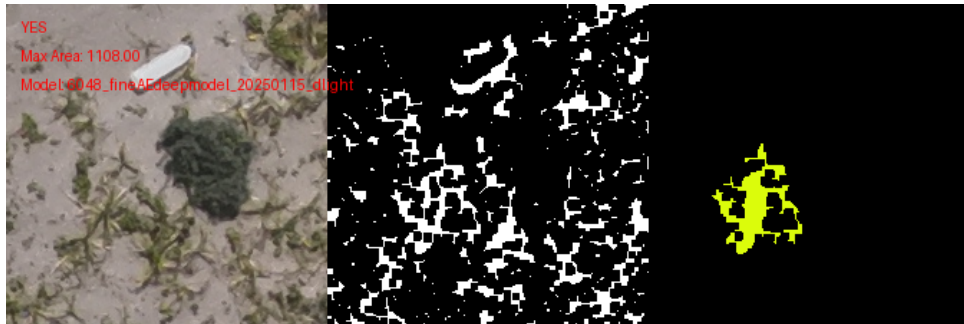


(b) rmodel によるノイズ除去及びクロージング画像

図 3.14: 正規化なし rmodel 検出結果

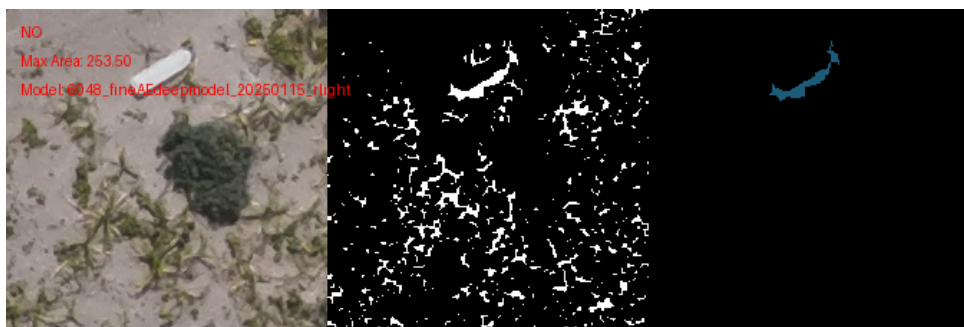


(a) dmodel によるノイズ除去画像

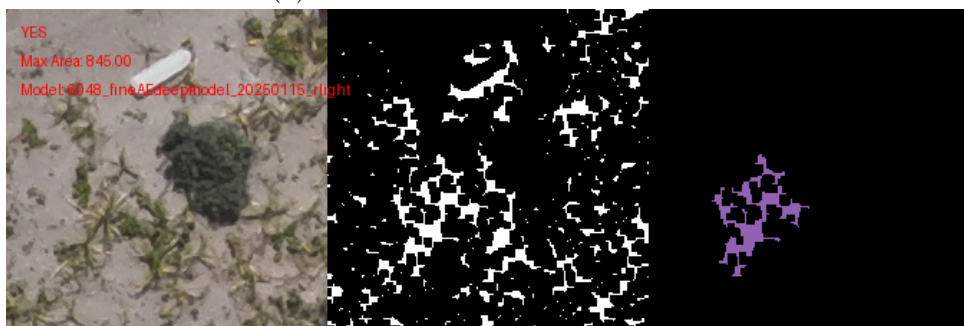


(b) dmodel によるノイズ除去及びクロージング画像

図 3.15: 正規化あり dmodel 検出結果



(a) rmodel によるノイズ除去画像



(b) rmodel によるノイズ除去及びクロージング画像

図 3.16: 正規化あり dmodel 検出結果

それぞれ 3 枚の画像が並んでいるが、左から順に入力画像、面積を検出する直前の画像、確認した最大面積の画像である。入力画像の左上には閾値が検出した最大面積を越えていれば YES が、越えていなければ NO が記されている。その下には最大面

積の大きさと MSE が最も小さかったモデル名が記されていて、今回の入力画像では light モデルにおける MSE が最小である。また、全ての画像に共通して物体部分の面積を適切に検出できていないことが確認できる。特にクロージングを行った画像に関しては物体ではない部分の面積を検出してしまい YES と判定してしまっている。これは AE が再構築した画像の再現性が低い部分をクロージングにより繋げてしまったためである。また、正規化を行っても結果が改善されていないことから、正規化は必要ないことが確認できる。

以上の結果から、システムに問題があるのではなく差分画像自体を修正する必要があると考えられる。

3.8 差分画像の精度向上

差分画像は出力画像によって決まるため、出力画像の精度を向上させることで差分画像への物体出現が多くなることが見込まれる。一般に学習枚数を増やすことで、モデルがより多くのパターンを学習することができるため、出力画像の精度が向上すると考えられる。しかし今回はアップデートのための学習を行っているため、必ずしも学習枚数が多ければ精度が向上するとは限らない。そのため学習枚数を減らしたモデルを構築し、そのモデルを用いて MSE の推移を確認する。また、物体検出に共通で失敗した画像を使用して MSE の推移を確認する。

学習枚数

これまで学習枚数は 6048 枚という数字にこだわってきた。これはアップデートをする前の昨年度のモデルを構築した際の学習枚数であり、一般に最初に学習するモデルには大量の画像が必要である。しかし、アップデート時にはそれを越すことはあまりない。例えば、Zhuang らの「A Comprehensive Review on Transfer Learning」[6]では、転移学習の手法について詳しく説明されており、転移学習では一般的に大規模なデータセットで事前学習を行い、その後、特定のタスクに対して小規模なデータセットでファインチューニングを行うことが効果的であると述べられている。この研究からもわかるように、初期学習には大量のデータが必要であるが、モデルのアップデート時にはそれを越えるデータが必要となることは稀である。

学習枚数の変更

学習枚数を減らしたデータセットを構築する。今回は 100 枚, 1000 枚, 2000 枚, 3000 枚, 4000 枚, 5000 枚のデータセットを構築する。結果は以下の通りである。

使用プログラム

- AE_train_img_move.ipynb

使用画像

- imgs/update_train_imgs_default/train_img_6048
- imgs/update_train_imgs_remove/train_img_6048

出力

指定の枚数のデータセットを構築する。

出力先

- imgs/update_train_imgs_default_mix
- imgs/update_train_imgs_remove_mix

モデルの構築

モデルの構築を行う。結果は以下の通りである。

使用プログラム

- AE_trainf.py

使用画像

- imgs/update_train_imgs_default_mix/train_img_n
 - imgs/update_train_imgs_remove_mix/train_img_n
- (n = 100, 1000, 2000, 3000, 4000, 5000)

出力

指定の枚数を学習したモデルが構築される。

出力モデル名

- models/fine_model_paths/n_fineAEdeepmodel_20250116_ddark.pth
 - models/fine_model_paths/n_fineAEdeepmodel_20250116_dlight.pth
 - models/fine_model_paths/n_fineAEdeepmodel_20250116_dwhite.pth
 - models/fine_model_paths/n_fineAEdeepmodel_20250116_rdark.pth
 - models/fine_model_paths/n_fineAEdeepmodel_20250116_rlight.pth
 - models/fine_model_paths/n_fineAEdeepmodel_20250116_rwhite.pth
- (n = 100, 1000, 2000, 3000, 4000, 5000)

MSE の推移の確認

構築したモデルを使用して MSE の推移を確認する。結果は以下の通りである。

使用プログラム

- AE_transition.ipynb

使用画像

- imgs/test_objects/detected_1_65.jpg

使用モデル（実行するとモデルが移動するため、移動後のパス）

- models/dmodels/n_fineAEdeepmodel_20250116_dlight.pth
 - models/rmodels/n_fineAEdeepmodel_20250116_rlight.pth
 - models/dmodels/6048_fineAEdeepmodel_20250115_dlight.pth
 - models/rmodels/6048_fineAEdeepmodel_20250115_rlight.pth
- (n = 100, 1000, 2000, 3000, 4000, 5000)

出力

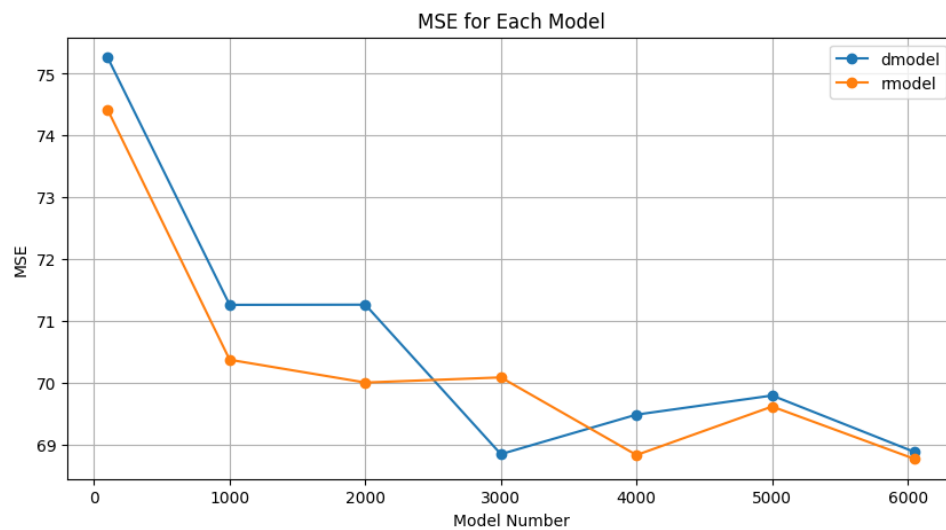


図 3.17: MSE の推移

図3.17は, 物体が検出できなかった画像に対する MSE の推移を示す. 横軸は学習枚数, 縦軸は MSE を示している. 学習枚数が増えるにつれて MSE が減少傾向にあることが確認できる. しかし, dmodel では 3000 枚, rmodel では 4000 枚で MSE が下がりきった後に上昇していることが確認でき, 過学習の可能性も示唆される. そのため, 再度それぞれのモデルで物体検出を行い, その結果を確認する.

物体検出

それぞれ適切な学習枚数のモデルを使用して物体検出を行う. 結果は以下の通りである.

使用プログラム

- `AE_inference.ipynb`

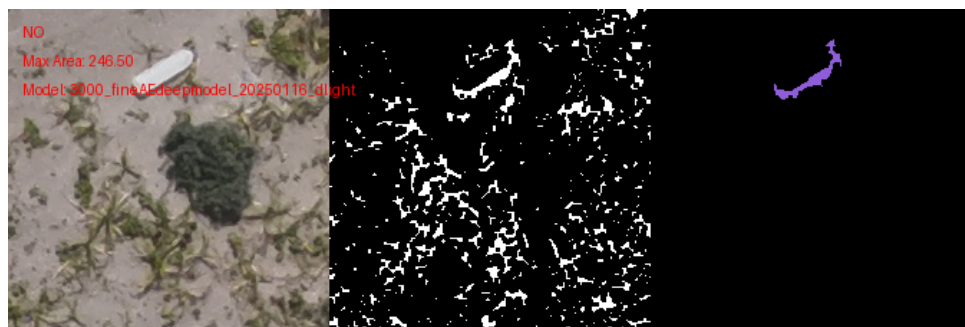
使用画像

- `imgs/test_objects/detected_1_65.jpg`

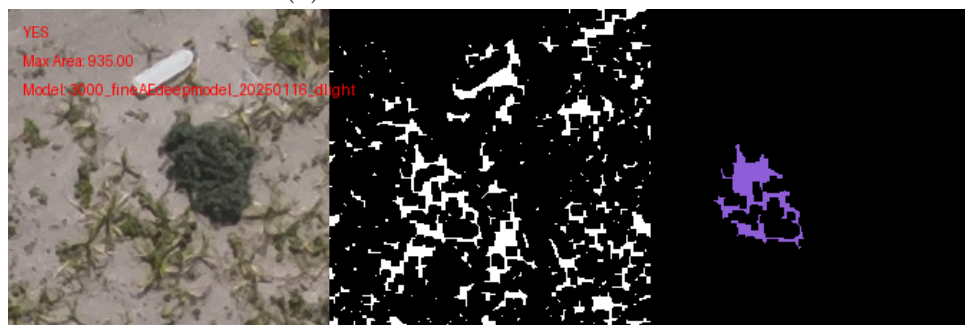
使用モデル

- `models/fine_model_paths/3000_fineAEdeepmodel_20250115_dlight.pth`
- `models/fine_model_paths/4000_fineAEdeepmodel_20250115_rlight.pth`

出力

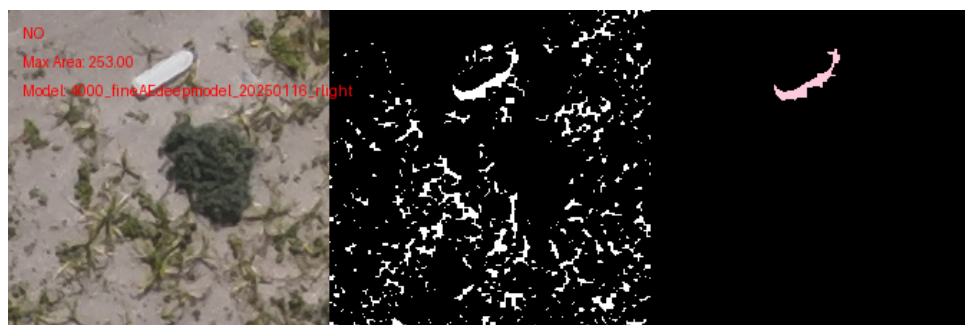


(a) dmodel による物体検出画像

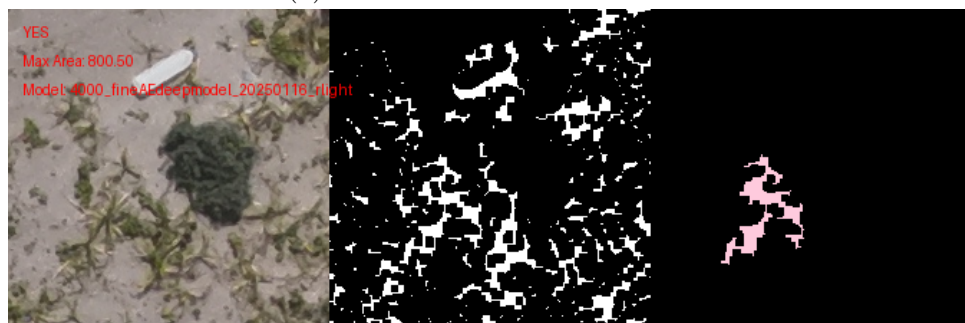


(b) dmodel によるクロージング物体検出画像

図 3.18: 3000dmodel 検出結果



(a) rmodel による物体検出画像



(b) rmodel によるクロージング物体検出画像

図 3.19: 4000rmodel 検出結果

図3.18は, 3000 枚学習した dmodel による物体検出結果を示す. 図3.19は, 4000 枚

学習した `rmodel` による物体検出結果を示す.

物体を検出することは出来なかった. これは, 学習枚数を増やすことで MSE が下がることが確認できたが, 物体検出の精度が向上することはなかったためである. また, 失敗の一因として, アップデート前のモデルの学習に使用した画像に問題があると考えられる. 特に昨年度は画像の枚数が限られていたため, 学習データを 16 倍にデータ拡張したことが過剰であり, モデルが実際のデータ分布を適切に学習できなかった可能性がある.

対応策として, 現在アップデートに使っている画像をデータ拡張を行わずに学習し, 次回撮影された画像でアップデートを行うことが考えられる. これは, 別日に撮影された画像で行う必要があると考えるが, 現状ではデータがないため, 今後の課題とする.

3.9 物体から鳥を抽出

物体の検出は失敗に終わった. しかし, 物体を検出できたと仮定して, 物体から鳥の画像を CLIP を使用して抽出することを試みる. 結果は以下の通りである.

使用プログラム

- `CLIP_bird.ipynb`

使用画像

- `imgs/test_objects_bird`

使用モデル

- `RN50x64(High Model)`

使用した言葉

本研究で使用した言葉は以下の通りである. これらの言葉は CLIP モデルに入力するために GoogleTranslator で英語に翻訳して使用された.

- `YES_bird`: 鳥の死骸が落ちている
- `NO_bird`: 鳥の死骸が落ちていない
- `NO_bird2`: 鳥の死骸ではなく物体が落ちている
- `NO_bird3`: 何も落ちていない

出力

1 枚の鳥の画像を含む 100 枚の画像を入力し, 鳥の画像を抽出する. 結果として, 鳥の画像が含まれる 11 枚の画像が抽出された.

F2-score の計算

- 混合行列の要素:
 - True Positives (TP): 1 枚
 - False Positives (FP): 10 枚
 - True Negatives (TN): 89 枚

- False Negatives (FN): 0 枚
- 精度 (Precision) と再現率 (Recall) を計算する.
 - $\text{Precision} = \frac{TP}{TP+FP} = \frac{1}{1+10} \approx 0.091$
 - $\text{Recall} = \frac{TP}{TP+FN} = \frac{1}{1+0} = 1.0$
- F2-score を計算する.
 - $\text{F2} = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.091 \cdot 1.0)}{4 \cdot 0.091 + 1.0} \approx 0.333$

F2-score はそこまで高い結果ではなかった. これは, 実際に検出システムが物体と判定した画像に対してではなく, 物体が検出されたと仮定したデータで行った結果であるため, F2-score の値を適切に評価することは難しく今後の課題とする.

4 結論

今年度は汎用的なモデルを構築することと, 物体画像から鳥の画像を自動で検出することを目標に研究を行ったが, 1 つ目の目標である汎用的なモデルの構築は失敗に終わった.

これは, 昨年度のモデルの学習に使用した画像に問題があると考えられる. そのため, 過剰にデータ拡張をせずに学習を行ったモデルをベースにアップデートを行うことで改善される可能性があり, 今後の課題とする. また, 性能の良いベースモデルが構築できたら, アップデートする部分を出力に近いデコード部分だけに変更しエンコード部分は固定することで, 学習時間の短縮に繋がると考えられる.

2 つ目の目標である物体画像から鳥の画像を自動で検出することに関しては, CLIP を用いることで鳥の画像を抽出することができたが, 実験に使用したデータが適切でなかったため, F2-score の値を適切に評価することは難しい. そのため, 適切なデータを使用し, 与える言葉もさらに調整した結果を評価することが必要である.

5 謝辞

本研究をまとめるにあたり, 多くの方々にご助力いただきましたこと, 心より感謝申し上げます. 指導教官として豊富な知識と経験の下, 適切な助言と終始多大なご指導を賜りました山形大学大学院理工学研究科脇克志教授に深く感謝いたしますとともに, 御礼申し上げます. 約 3 年間の間, 研究活動に携わらせていただいた中で, 興味を持っていた画像処理と深層学習について学ぶことができ, 非常に大きな力となりました. また, 学会等に参加させていただいたことは, 自分自身の価値観を広げる貴重な経験となりました.

次に, 副指導教員として深層学習および Python に関する様々な知識とご助言を賜りました, 山形大学理工学研究科 Yago Diez 准教授に厚くお礼申し上げます. モデルの作成や仕組みについて深く学ばせていただき, 多くのご指導を賜りました.

さらに, 研究に関しての画像提供をはじめとする研究協力をいただきました, 山形県企業局酒田電気水道事務所 (平田浄水場) 渡邊弘章様, 鈴木健市様, 木村俊之様, 齋藤洋美様, 村上浩様, 荒井隆男様, 大友浩一様, 鈴木旺様, 今野朗様, 梅田貴之様に深く感謝申し上げます. 風力発電所での調査見学や月例報告会でも多くのご意見をいただき, 非常に感謝しております.

また, 同じ研究室のゼミ生として, 切磋琢磨しながら研究活動を行ってきた, 梅津魁

秀さん, 馬場優弥さん, 百井海星さんに厚くお礼申し上げます. お二人と一緒に活動できたおかげで, 自分自身も楽しく活動しながらも成長することができました.

最後になりますが, 大学生活において自分に携わってくださった全ての皆様, これまで自分を育ててくれた両親に感謝いたします.

6 使用したデータの説明

本章では, 研究で使ったデータについて説明する. 始めに実験に使ったプログラミング環境とライブラリについて述べ, その後, 実験に使ったファイルとディレクトリの構造を示す. さらに, 使った画像データとモデルデータについて詳細に説明する.

6.1 プログラミング環境と使ったライブラリ

実験は Python を用いて行った. 主なプログラミング環境と使ったライブラリは以下の通りである.

- **Python 3.10.12**: プログラミング言語として使用.
- **collections.defaultdict** (標準ライブラリ): デフォルト値を持つ辞書に使用.
- **csv** (標準ライブラリ): CSV ファイルの操作に使用.
- **cv2** (4.10.0.84): 画像処理に使用.
- **datetime** (標準ライブラリ): 日付と時間の操作に使用.
- **deep_translator** (1.11.4): テキストの翻訳に使用.
- **glob** (標準ライブラリ): ファイルパスのパターンマッチングに使用.
- **matplotlib.pyplot** (3.10.0): グラフの描画に使用.
- **numpy** (1.26.3): 数値計算に使用.
- **open_clip** (2.29.0): CLIP モデルの使用.
- **open_clip.tokenizer** (2.29.0): テキストのトークン化に使用.
- **os** (標準ライブラリ): OS 機能へのアクセスに使用.
- **PIL.Image** (11.0.0): 画像操作に使用.
- **PIL.ImageDraw** (11.0.0): 画像に描画するために使用.
- **PIL.ImageFont** (11.0.0): 画像にフォントを使用するために使用.
- **PIL.ImageOps** (11.0.0): 画像の操作に使用.
- **random** (標準ライブラリ): 乱数生成に使用.
- **re** (標準ライブラリ): 正規表現操作に使用.
- **shutil** (標準ライブラリ): 高レベルのファイル操作に使用.

- **sklearn.cluster.KMeans** (1.6.0): K-means クラスタリングに使用.
- **sklearn.decomposition.PCA** (1.6.0): 主成分分析に使用.
- **sklearn.preprocessing.LabelEncoder** (1.6.0): ラベルエンコーディングに使用.
- **torch** (2.5.1+cu118): 深層学習モデルの構築と訓練に使用.
- **torch.nn** (2.5.1+cu118): ニューラルネットワークの構築に使用.
- **torch.optim** (2.5.1+cu118): 最適化アルゴリズムに使用.
- **torch.utils.data.DataLoader** (2.5.1+cu118): データローダーの操作に使用.
- **torch.utils.data.Dataset** (2.5.1+cu118): データセットの操作に使用.
- **torchvision.transforms** (0.20.1+cu118): 画像の前処理に使用.
- **tqdm** (4.67.1): プログレスバーの表示に使用.

6.2 想定環境

```

/
2023_main.py
2024_main.py
AE_clip.ipynb
AE_inference.ipynb
AE_make_dataset.ipynb
AE_pca_kmeans.ipynb
AE_train_img_move.ipynb
AE_train.py
AE_trainf.py
AE_transition.py
CLIP_bird.ipynb
imgs/
models/

```

6.3 画像データ

- **imgs/for_update_img**
説明: 2024 年 9 月 3 日に撮影した 3000×4000 ピクセルの画像が 156 枚含まれている.
- **imgs/make_pca_img1000**
説明: A_train_img と重複しない画像が 1000 枚ずつ, 計 4000 枚含まれている.
- **imgs/old_all_img/train_img_6048/6048/A_train_img**
説明: 昨年使用した 224×224 ピクセルの画像が背景ごとに分類されずに 1512 枚ずつ, 計 6048 枚含まれている.

- **imgs/old_train_img3/train_img_6048/6048**

- **new_dark/**: 6048 枚
- **new_light/**: 6048 枚
- **new_white/**: 6048 枚

説明: 昨年学習に使用した背景ごとに分類された 224×224 ピクセルの画像がそれぞれ含まれている.

- **imgs/test_img**

説明: 2024 年 9 月 3 日に撮影した for_update_img には含まれていない 3000×4000 ピクセルの画像が 4 枚含まれている.

- **imgs/test_objects**

説明: test_img を 224×224 ピクセルに分割した画像の中から, 物体が存在する画像が 13 枚含まれている.

- **imgs/test_objects_bird**

説明: test_objects と鳥の死骸の画像 1 枚と無作為に抽出した画像 86 枚の計 100 枚が含まれている.

- **imgs/update_img_remove/**

- **dark_object/**:306 枚
- **dark/**:4973 枚
- **light_object/**:1703 枚
- **light/**:7518 枚
- **white_object/**:1925 枚
- **white/**:3064 枚

説明: update_img (今後製作される) を背景ごとに物体が存在する画像としない画像に分類した.

6.4 モデルデータ

- **models/2023models/**

- **AEmodel_dark_green20230927.pth**
- **AEmodel_light_green20230927.pth**
- **AEmodel_white20230927.pth**
- **AEmodel_paved_ground20230927.pth**

説明: 昨年構築した背景ごとに 224×224 ピクセルの画像を 6048 枚学習させたモデル.

6.5 使用プログラム

2023_main.py

出力までの流れ

1. 画像の分割: 入力画像を 224×224 ピクセルの小さな画像に分割した.
2. 物体検出の実行: 分割された画像を AE を使用して, 物体が存在するかどうかを判定した.
 - AE モデルを使用して, 入力画像を再構築する.
 - 再構築された画像と元の画像の差分を計算し, 差分画像を生成する.
 - 再構築された画像と元の画像の MSE を計算し, MSE が最も小さいモデルの差分画像を採用する.
 - 差分画像から輪郭を抽出し, 最大面積の輪郭を計算する.
 - 最大面積が閾値を超える場合, 物体が存在すると判定する.
3. 結果の保存: 物体が存在すると判定された画像を保存した.
4. 評価指標の計算: 混合行列を作成し, F2-score を計算した.

2024_main.py

出力までの流れ

2023_main.py とモデルの読み込み以外同じため省略.

AE_clip.ipynb

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. モデルの読み込み: 事前に訓練された CLIP モデル (RN50x64 および RN50) を読み込む.
3. 翻訳: 使用する日本語の説明文を GoogleTranslator で英語に翻訳する.
4. 画像のコピー: 指定されたフォルダからランダムに画像を選択し, テスト用フォルダにコピーする.
5. テスト画像の予測とプロット: テスト用にコピーした画像に対して CLIP モデルを用いて予測を行い, 結果をプロットして保存する.
6. 画像の予測: メインの画像に対して CLIP モデルを用いて予測を行う.
7. 画像の分類と保存: 予測結果に基づいて画像を分類し, 結果を保存する.
8. フォルダのコピーと整理: 分類結果のフォルダをコピーし, 不要なディレクトリを削除する.

9. 画像の移動: 物体画像を含むフォルダから画像を移動し, 物体の割合を計算する.
10. 画像のコピーと反転処理: 画像をコピーし, 6048 枚になるように, 必要に応じて反転や回転処理を行う.

AE_inference.ipynb

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. モデルの定義と読み込み: 指定されたパスからモデルを読み込む.
3. 画像の分割: 入力画像を 224×224 ピクセルに分割する.
4. 差分画像の生成: モデルを使用して入力画像と出力画像の差分画像を生成する.
5. 二値化とノイズ除去: 差分画像を二値化し, ノイズ除去を行う.
6. 輪郭検出と最大面積の算出: 二値化画像から輪郭を検出し, 最大面積を算出する.
7. 結果の保存: 差分画像, 二値化画像, 輪郭画像, 入力画像, 出力画像を保存し, 結果をプロットして保存する.

AE_make_dataset.ipynb

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. モデルの読み込み: 事前に訓練された AE モデルを読み込む.
3. 画像の読み込み: 指定されたディレクトリから画像を読み込む.
4. 画像の分割: 読み込んだ画像を 224×224 ピクセルの小さな画像に分割する.
5. **AE** による特徴抽出: 分割された画像を AE モデルに通し, 特徴ベクトルを抽出する.
6. **PCA** による次元削減: 抽出された特徴ベクトルに対して PCA を行い, 次元削減を行う.
7. **PCA** 結果のプロット: PCA による次元削減結果をプロットし, 視覚的に確認する.
8. **K-means** クラスタリング: 次元削減された特徴ベクトルに対して K-means クラスタリングを行い, 画像をクラスタに分類する.
9. クラスタリング結果のプロット: K-means クラスタリングの結果をプロットし, 各クラスタの分布を視覚的に確認する.
10. 画像の移動: クラスタリング結果に基づいて, 画像を新しいフォルダに移動する.

AE_pca_kmeans.ipynb

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. モデルの読み込み: 事前に訓練された AE モデルを読み込む.
3. 画像の読み込み: 指定されたディレクトリから画像を読み込む.
4. **AE** による特徴抽出: 読み込んだ画像を AE モデルに通し, 特徴ベクトルを抽出する.
5. **PCA** による次元削減: 抽出された特徴ベクトルに対して PCA を行い, 次元削減を行う.
6. **PCA** 結果のプロット: PCA による次元削減結果をプロットし, 視覚的に確認する.
7. **K-means** クラスタリング: 次元削減された特徴ベクトルに対して K-means クラスタリングを行い, 画像をクラスタに分類する.
8. クラスタリング結果のプロット: K-means クラスタリングの結果をプロットし, 各クラスタの分布を視覚的に確認する.

AE_train.py

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. データの準備: 指定されたディレクトリから画像データを読み込み, 学習用と評価用に分割する.
3. データセットの構築: 画像データを読み込み, PyTorch のデータセットとして準備する.
4. データローダーの構築: 学習用と評価用のデータローダーを構築する.
5. モデルの定義: 深層 AE モデルを定義する.
6. 最適化手法と誤差関数の設定: モデルの最適化手法として Adam を使用し, 誤差関数として MSELoss を設定する.
7. 学習の実行: 指定されたエポック数だけモデルの学習を行う. 学習中に最も良いモデルを保存する.
8. 評価の実行: 学習後, 評価用データを用いてモデルの性能を評価する.
9. 結果の保存: 学習と評価の損失をプロットし, 画像として保存する. また, 損失の推移を CSV ファイルに保存する.

AE_train_img_move.ipynb

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. すべてのサブフォルダを取得: glob を使用して, ソースディレクトリ内のすべてのサブフォルダを取得する.
3. サブフォルダ内の画像ファイルを取得: 各サブフォルダ内のすべての画像ファイルを取得する.
4. 画像ファイルをランダムに選択: 指定された数の画像ファイルをランダムに選択する.
5. コピー先のサブフォルダを構築: 選択された画像ファイルをコピーするためのサブフォルダを構築する.
6. 画像ファイルをコピー: 選択された画像ファイルをコピー先のサブフォルダにコピーする.
7. データセットの構築: 指定された数の画像ファイルを含むデータセットを構築する.
8. 異なるサイズのデータセットを構築: 異なるサイズのデータセットを構築する.

AE_trainf.py

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. データの準備: 指定されたディレクトリから画像データを読み込み, 学習用と評価用に分割する.
3. データセットの構築: 画像データを読み込み, PyTorch のデータセットとして準備する.
4. データローダーの構築: 学習用と評価用のデータローダーを構築する.
5. モデルの定義: 深層 AE モデルを定義する.
6. モデルの読み込み: 事前に保存されたモデルの重みを読み込む.
7. 最適化手法と誤差関数の設定: モデルの最適化手法として Adam を使用し, 誤差関数として MSELoss を設定する.
8. 学習の実行: 指定されたエポック数だけモデルの学習を行う. 学習中に最も良いモデルを保存する.
9. 評価の実行: 学習後, 評価用データを用いてモデルの性能を評価する.
10. 結果の保存: 学習と評価の損失をプロットし, 画像として保存する. また, 損失の推移を CSV ファイルに保存する.

AE_transition.ipynb

出力までの流れ

1. 必要なライブラリのインポート: 必要なライブラリをインポートする.
2. シードの固定: 再現性を確保するために, 乱数シードを固定する.
3. モデルの定義と読み込み: AE モデルを定義し, 事前に保存されたモデルの重みを読み込む.
4. 画像の分割: 画像を指定されたサイズに分割する.
5. モデルの作成: モデルを作成し, 読み込んだ重みを適用する.
6. **MSE** の計算とプロット: 各モデルに対して分割された画像の MSE を計算し, 結果をプロットする.

CLIP_bird.ipynb

出力までの流れ

1. シードの固定: 再現性を確保するために, 乱数シードを固定する.
2. モデルの読み込み: 事前に訓練された CLIP モデル RN50x64 を読み込む.
3. 翻訳: 使用する日本語の説明文を GoogleTranslator で英語に翻訳する.
4. 画像の予測: メインの画像に対して CLIP モデルを用いて予測を行う.
5. 結果の表示: 分類結果を表示し, 特定の条件に一致する画像の数をカウントする.

6.6 モデル構築

始めに, モデルを構築するときのパラメータについて説明する.

シードの固定

再現性を確保するために, 乱数シードを固定する. これにより, 同じデータセットとパラメータを使用した場合に, 同じ結果が得られるようにする.

データの準備

データは指定されたディレクトリから読み込み, 学習用と評価用に分割する. データは 8:2 の割合で分割され, 学習用データと評価用データとして使用する.

モデルの定義

DeepAutoencodermodel を使用する. このモデルは, エンコーダとデコーダの 2 つの部分から構成されており, 入力データを圧縮し, その圧縮されたデータから元のデータを再構築することを目的としている. 詳細は図2.5を参照.

学習パラメータ

学習に使用するパラメータは以下の通り.

- エポック数: モデルの学習は最大で 1000 エポック行う.
- デバイス: GPU (利用可能な場合), それ以外は CPU.
- 最適化手法: モデルの最適化には Adam オプティマイザを使用する.
- 誤差関数: モデルの誤差関数として平均二乗誤差 (MSELoss) を使用する.

収束条件

学習の収束条件として, 連続して 30 エポック以上改善が見られない場合に学習を停止する設定を行う. この条件により, 過学習を防ぎつつ効率的な学習を行うことができる.

バッチサイズ

学習に使用するバッチサイズは, 学習データの枚数に応じて以下のように設定する.

- 学習データが 100 枚の場合: バッチサイズは 8
- 学習データが 1000 枚の場合: バッチサイズは 32
- 学習データが 2000 と 3000 枚の場合: バッチサイズは 64
- 学習データが 4000 と 5000 枚の場合: バッチサイズは 128
- 学習データが 6048 枚の場合: バッチサイズは 256

学習と評価の実行

学習と評価のプロセスは以下の通り. 学習中に最も良いモデルを保存し, 学習と評価の損失をプロットする.

1. 学習:
 - モデルを学習用データで訓練し, 各エポックごとに損失を計算する.
 - 損失が最も少ないモデルを保存する.
2. 評価:
 - 評価用データを使用してモデルの性能を評価する.
 - 学習と評価の損失をプロットし, 画像として保存する.

これにより, モデルの学習プロセスとパラメータ設定について詳細に説明することができる.

以下に構築したモデルの名前と学習曲線を示す.

モデルの名前と学習曲線

- `models/6048_AEdeepmodel_20241224_A_train_img.pth`

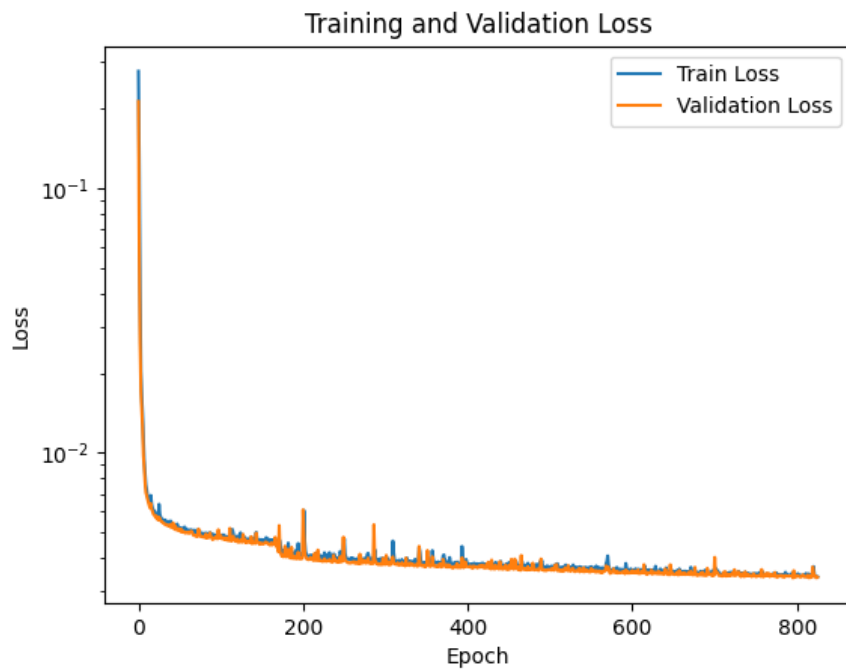


図 6.1: 学習曲線

- `models/old_models6048/20241225_new_dark.pth`

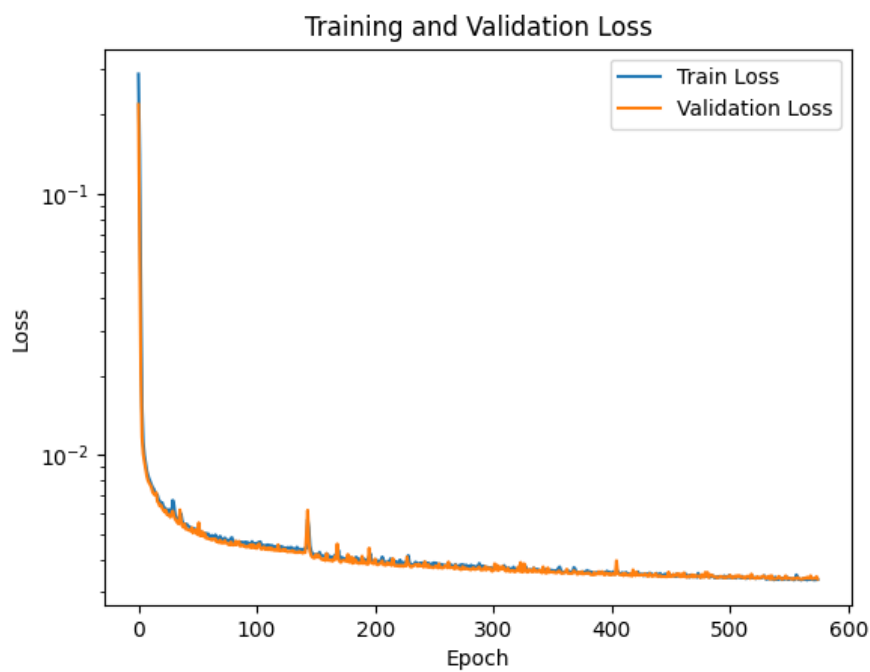


図 6.2: 学習曲線

- `models/old_models6048/20241225_new_light.pth`

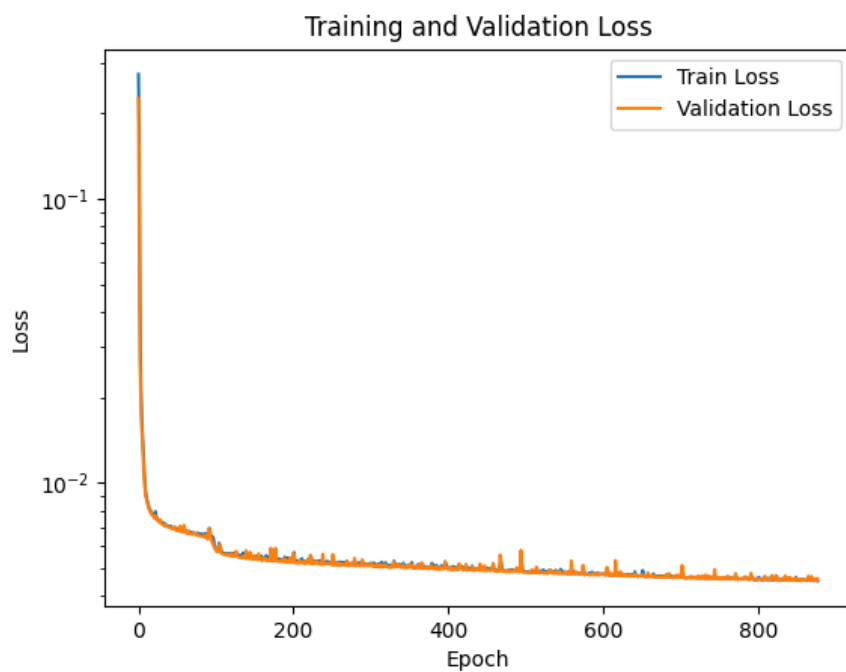


図 6.3: 学習曲線

- `models/old_models6048/20241225_new_white.pth`

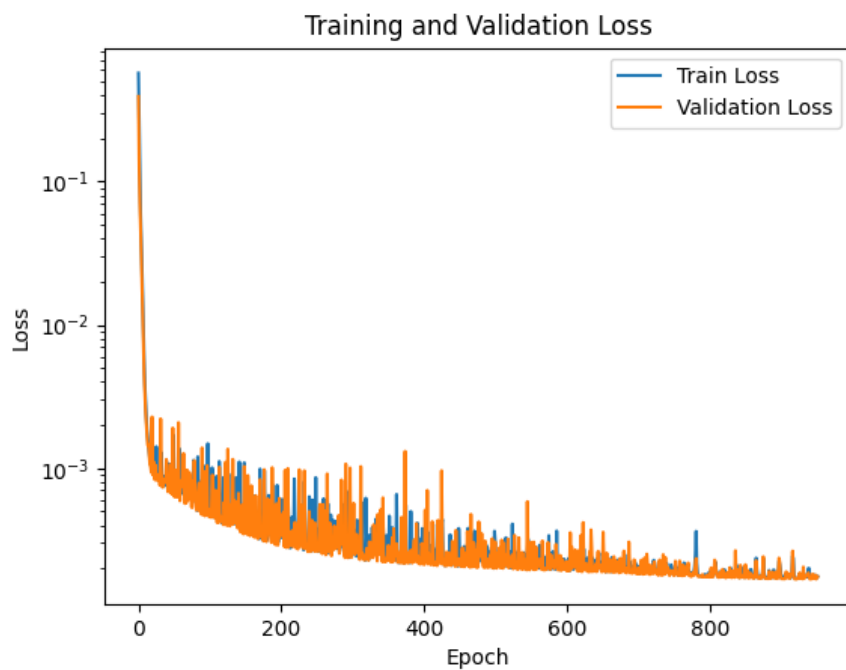


図 6.4: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_ddark.pth`

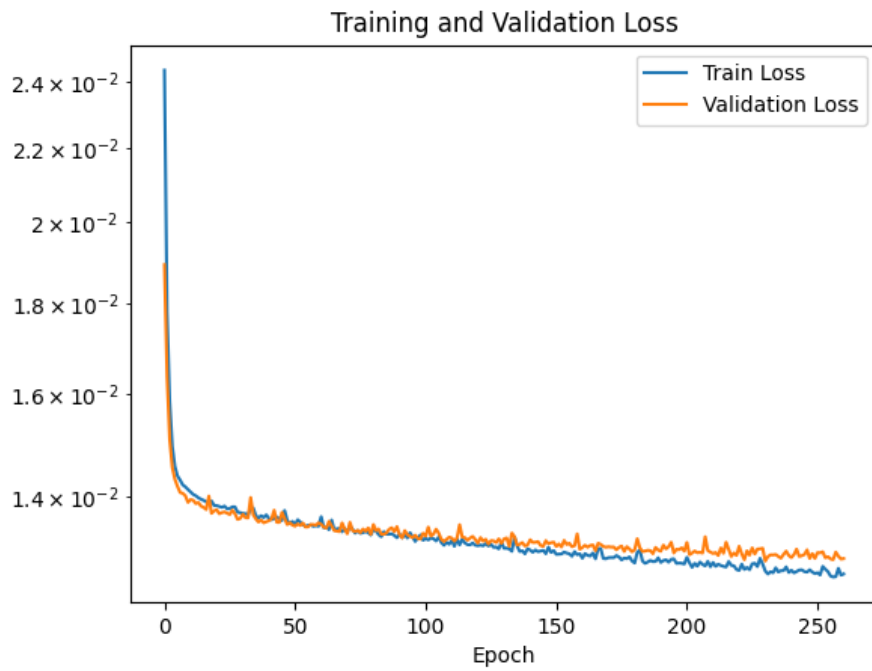


図 6.5: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rdark.pth`

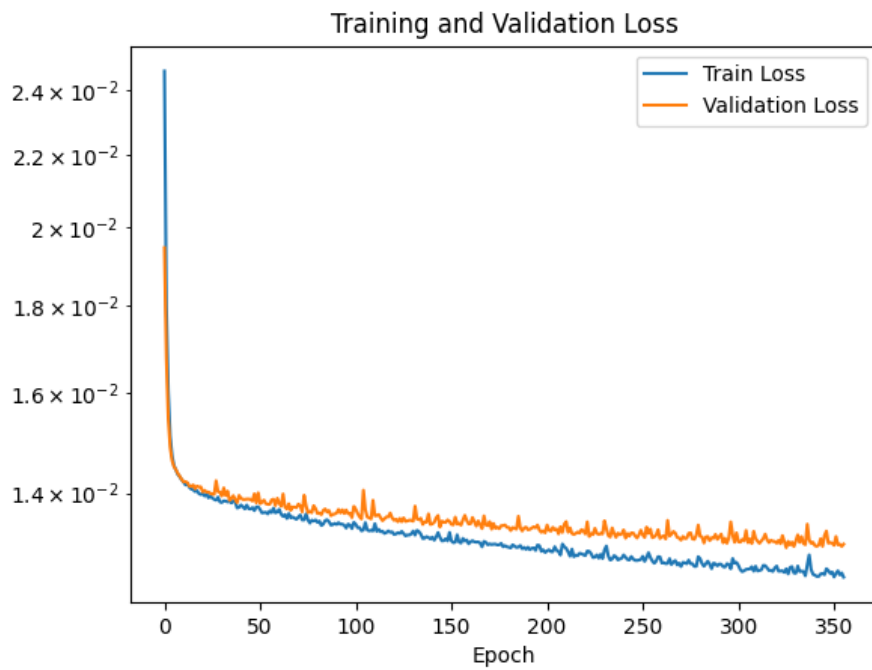


図 6.6: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_ddark.pth`

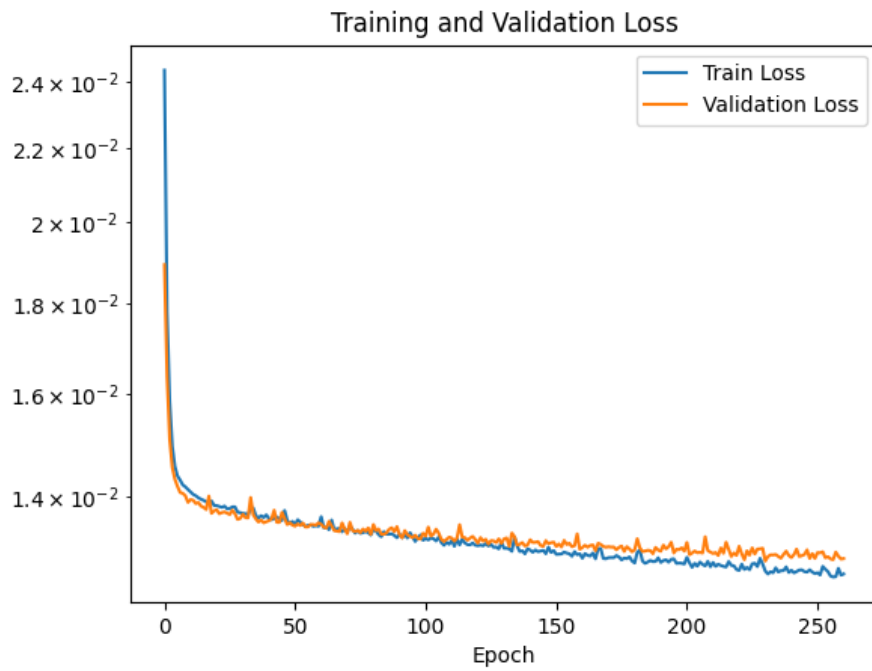


図 6.7: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rdark.pth`

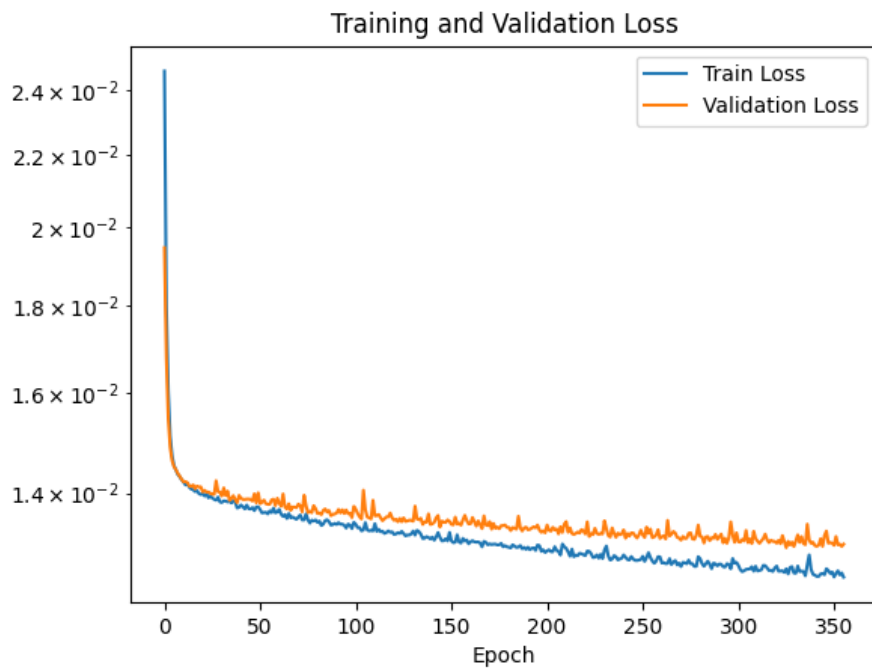


図 6.8: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_ddark.pth`

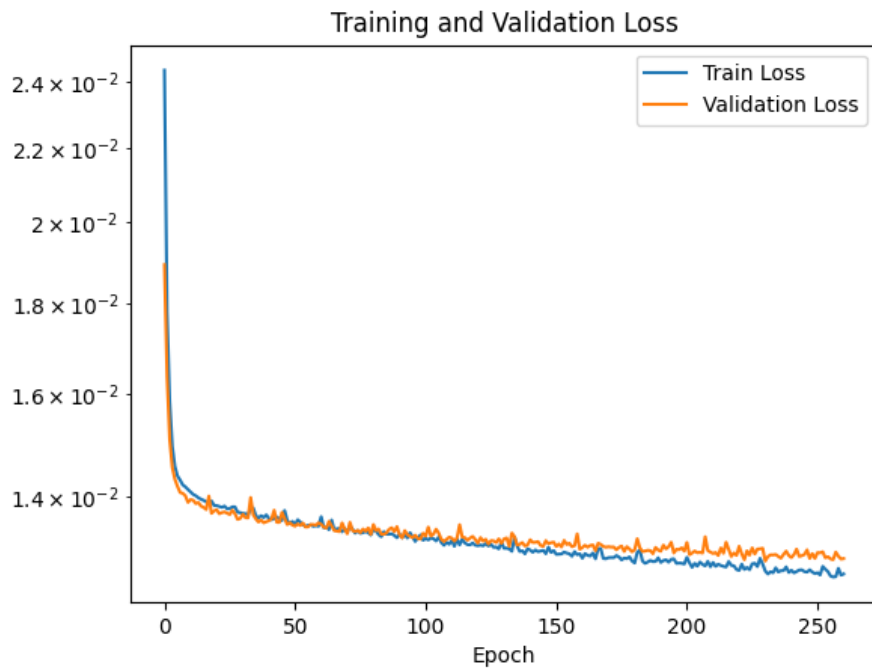


図 6.9: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rdark.pth`

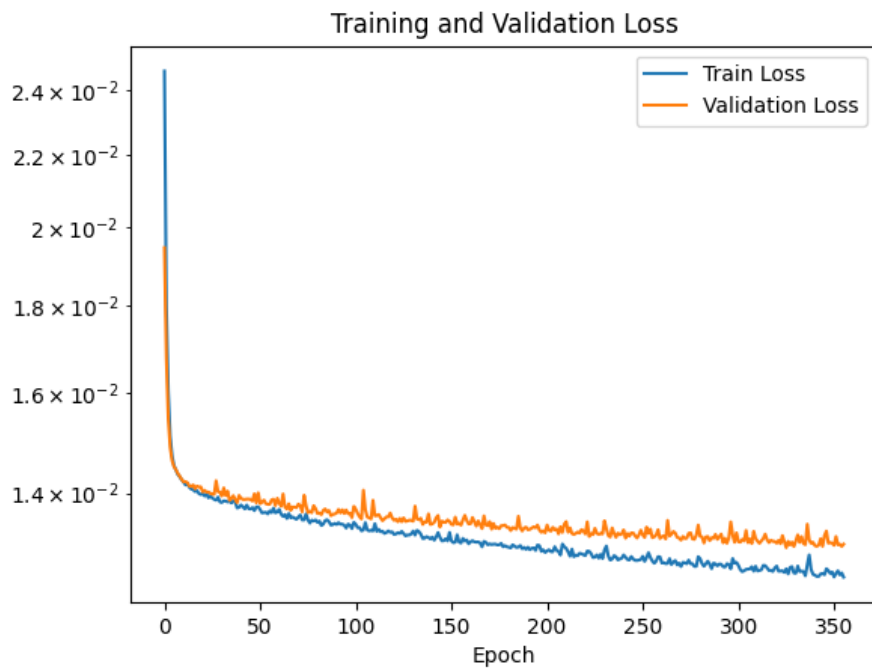


図 6.10: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_dlight.pth`

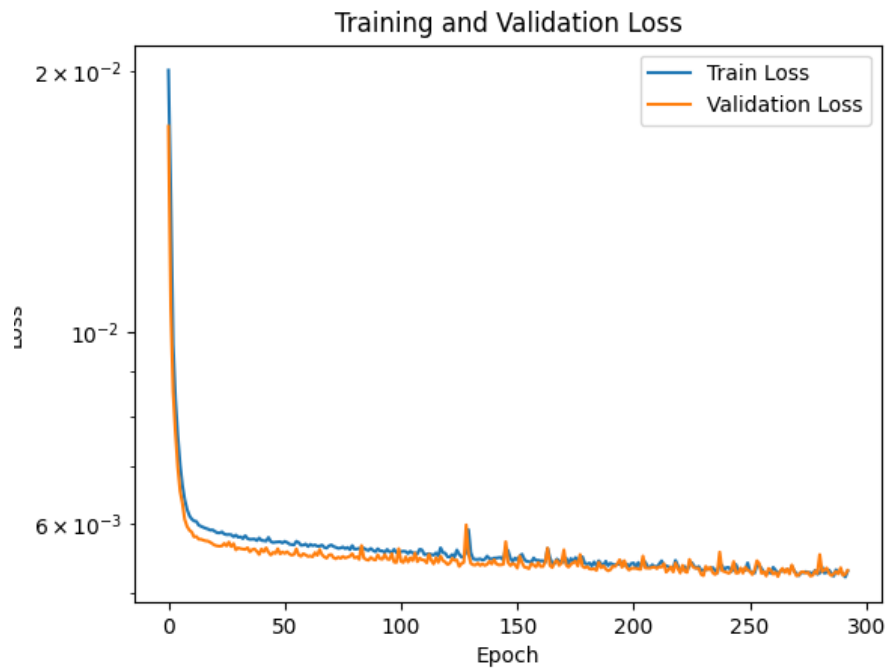


図 6.11: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rlight.pth`

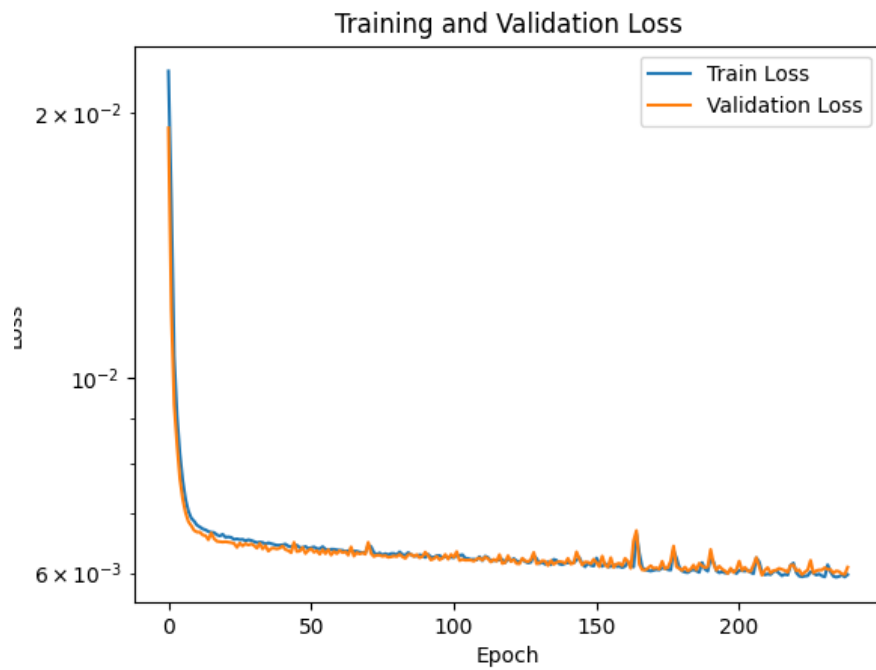


図 6.12: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_dwhite.pth`



図 6.13: 学習曲線

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rwhite.pth`

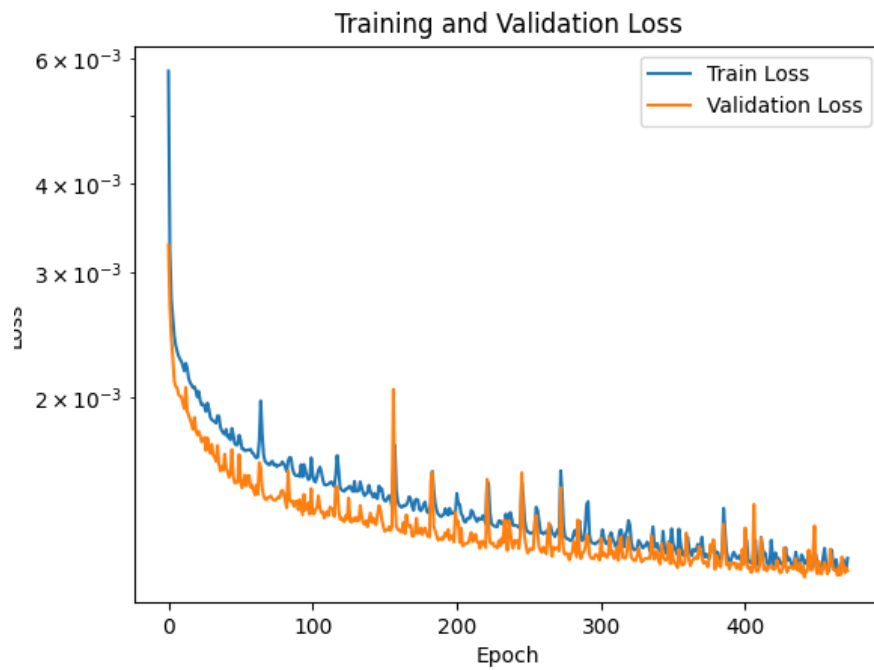


図 6.14: 学習曲線

- `models/dmodels/5000_fineAEdeepmodel_20250116_ddark.pth`



図 6.15: 学習曲線

- `models/rmodels/5000_fineAEdeepmodel_20250116_rdark.pth`



図 6.16: 学習曲線

- `models/dmodels/5000_fineAEdeepmodel_20250116_dlight.pth`

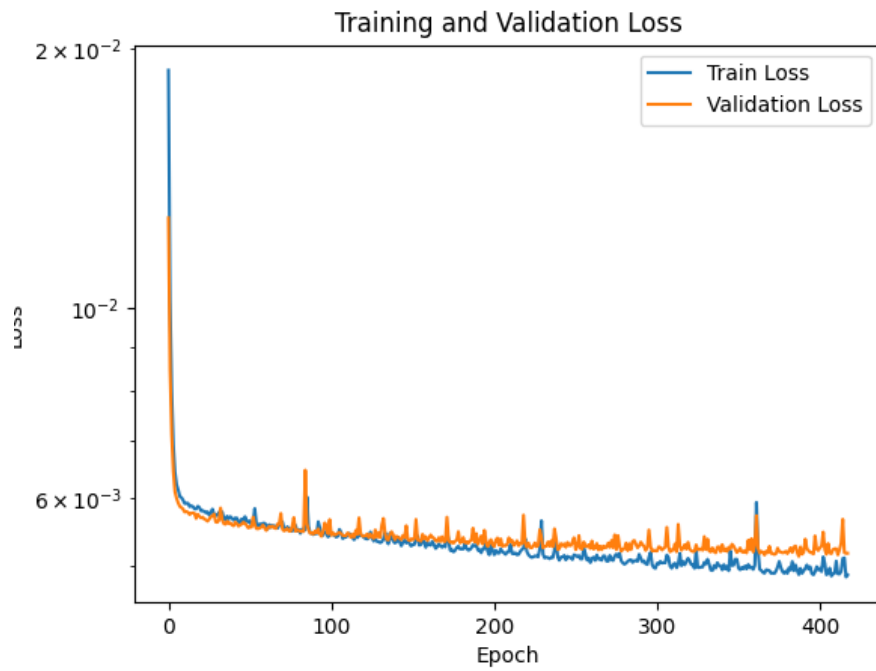


図 6.17: 学習曲線

- `models/rmodels/5000_fineAEdeepmodel_20250116_rdark.pth`

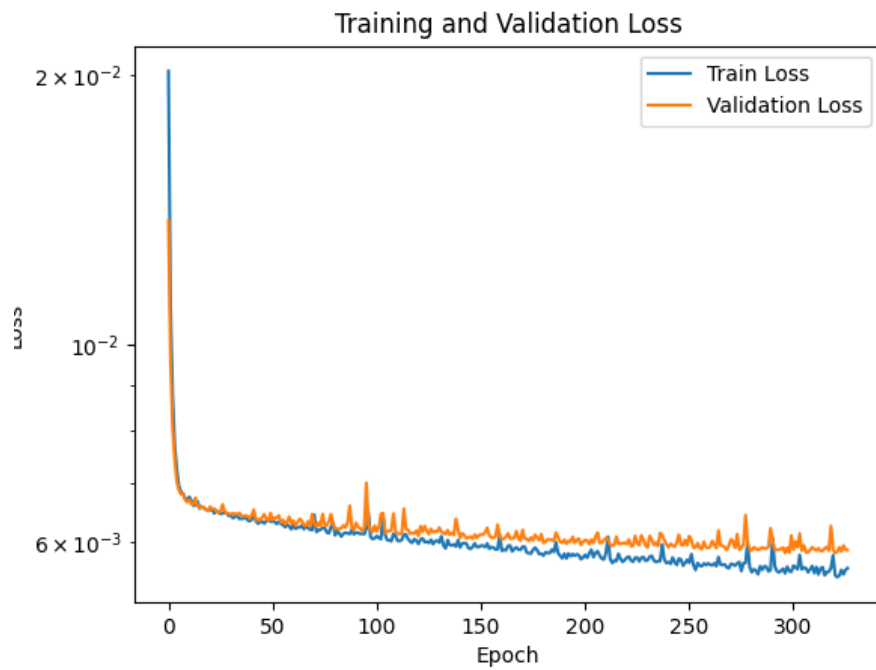


図 6.18: 学習曲線

- `models/dmodels/5000_fineAEdeepmodel_20250116_dwhite.pth`



図 6.19: 学習曲線

- `models/rmodels/5000_fineAEdeepmodel_20250116_rwhite.pth`

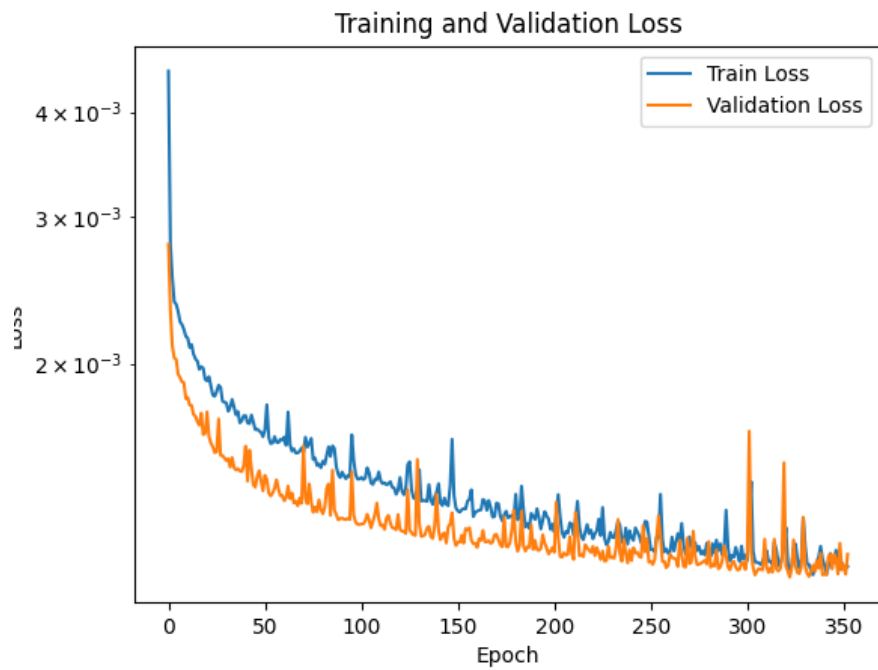


図 6.20: 学習曲線

- `models/dmodels/4000_fineAEdeepmodel_20250116_ddark.pth`

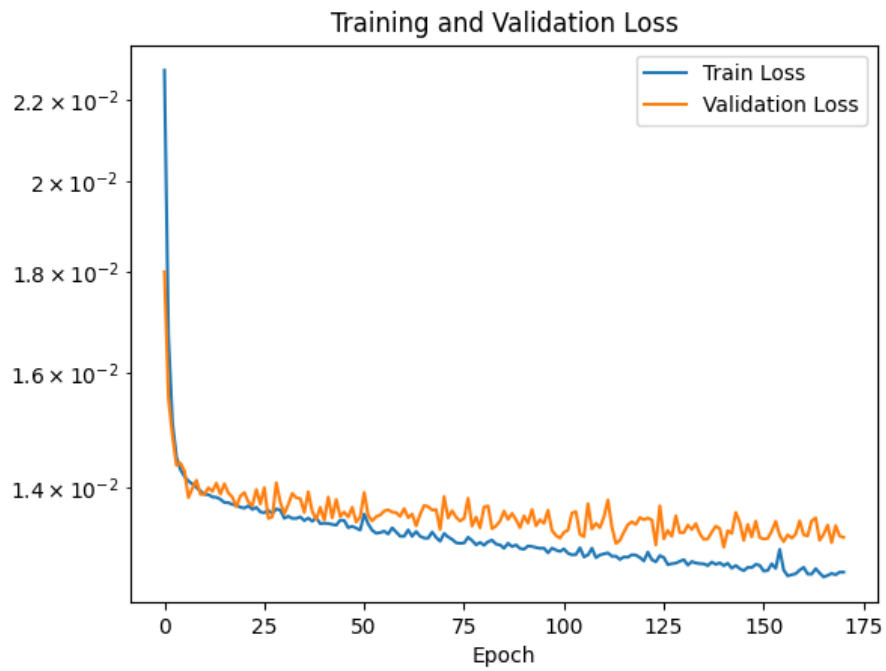


図 6.21: 学習曲線

- `models/rmodels/4000_fineAEdeepmodel_20250116_rdark.pth`

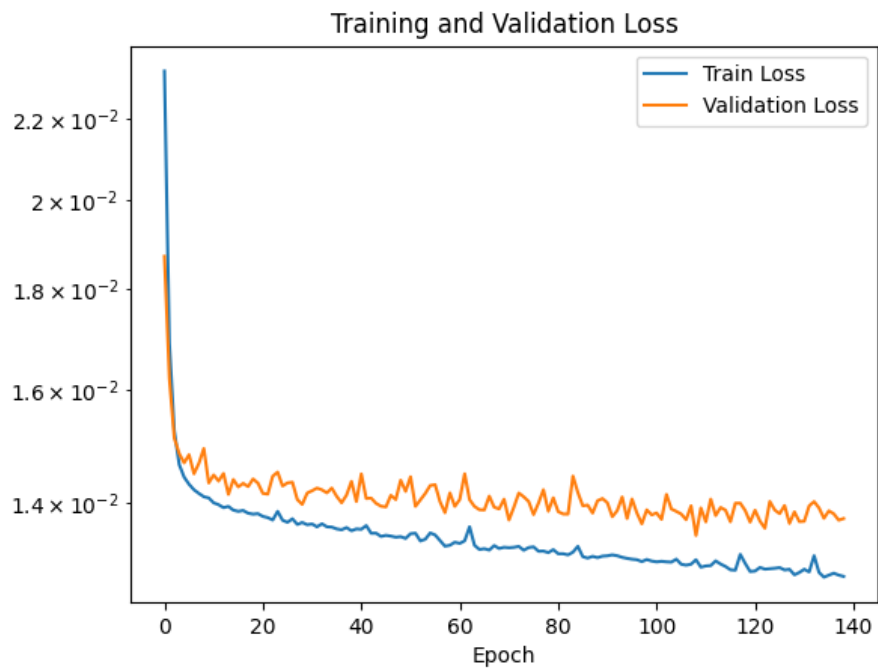


図 6.22: 学習曲線

- `models/dmodels/4000_fineAEdeepmodel_20250116_dlight.pth`



図 6.23: 学習曲線

- `models/rmodels/4000_fineAEdeepmodel_20250116_rdark.pth`

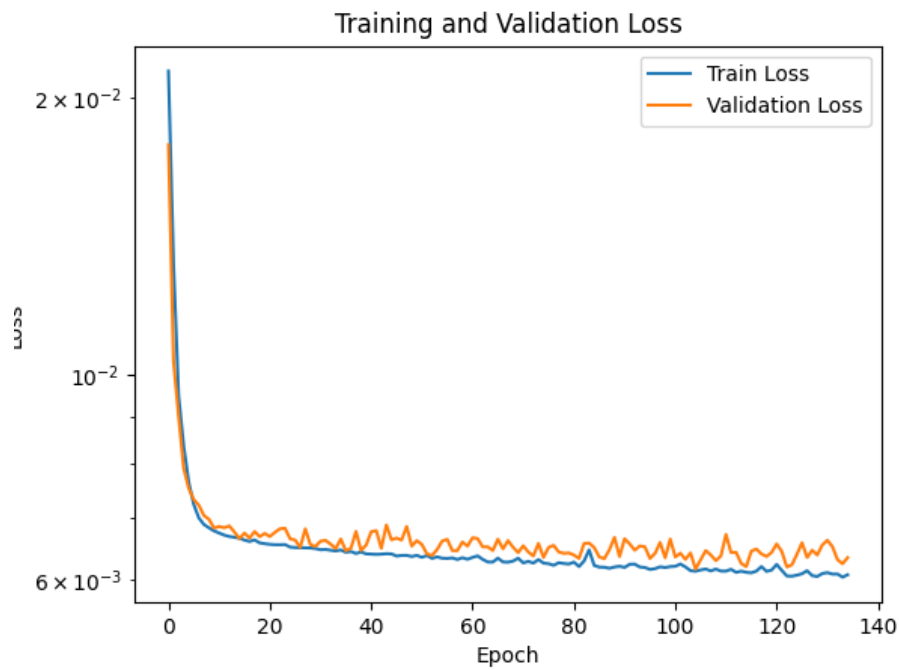


図 6.24: 学習曲線

- `models/dmodels/4000_fineAEdeepmodel_20250116_dwhite.pth`

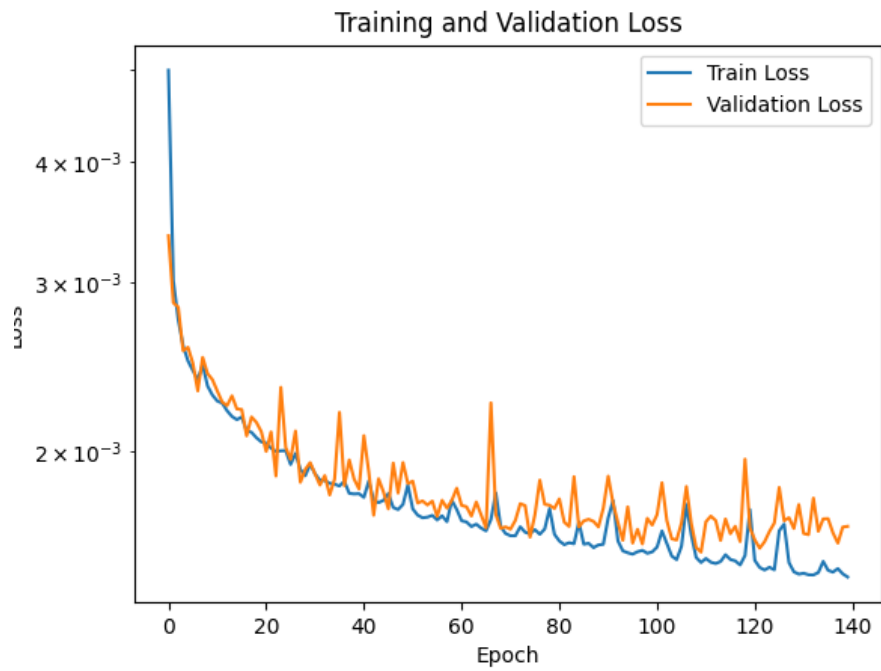


図 6.25: 学習曲線

- `models/rmodels/4000_fineAEdeepmodel_20250116_rwhite.pth`

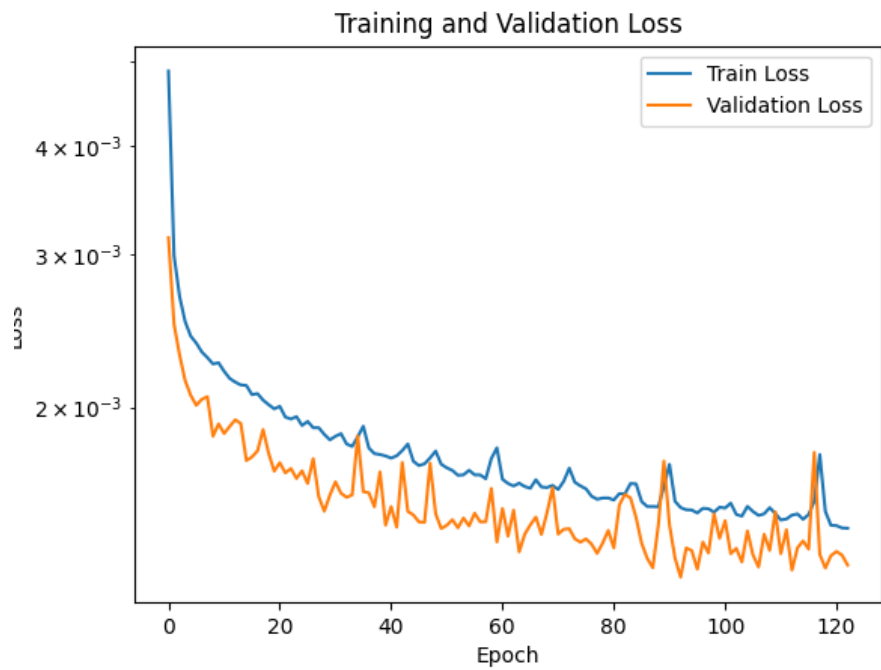


図 6.26: 学習曲線

- `models/dmodels/3000_fineAEdeepmodel_20250116_ddark.pth`

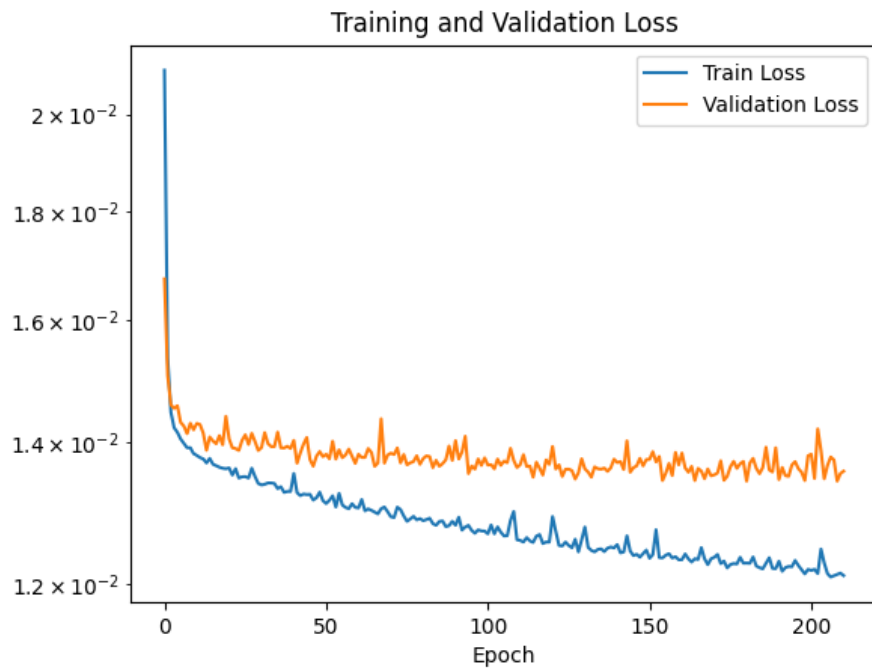


図 6.27: 学習曲線

- `models/rmodels/3000_fineAEdeepmodel_20250116_rdark.pth`

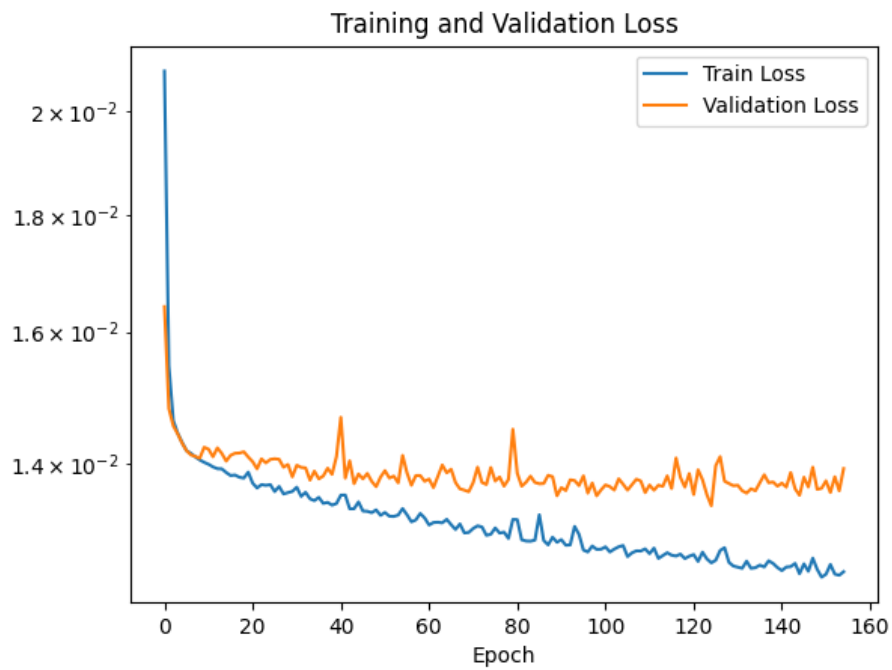


図 6.28: 学習曲線

- `models/dmodels/3000_fineAEdeepmodel_20250116_dlight.pth`

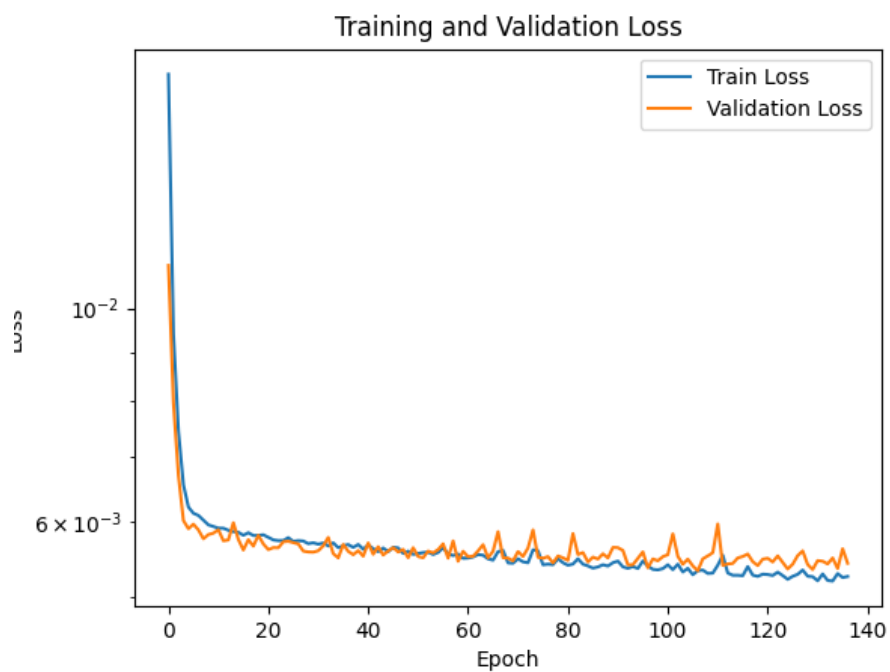


図 6.29: 学習曲線

- `models/rmodels/3000_fineAEdeepmodel_20250116_rdark.pth`

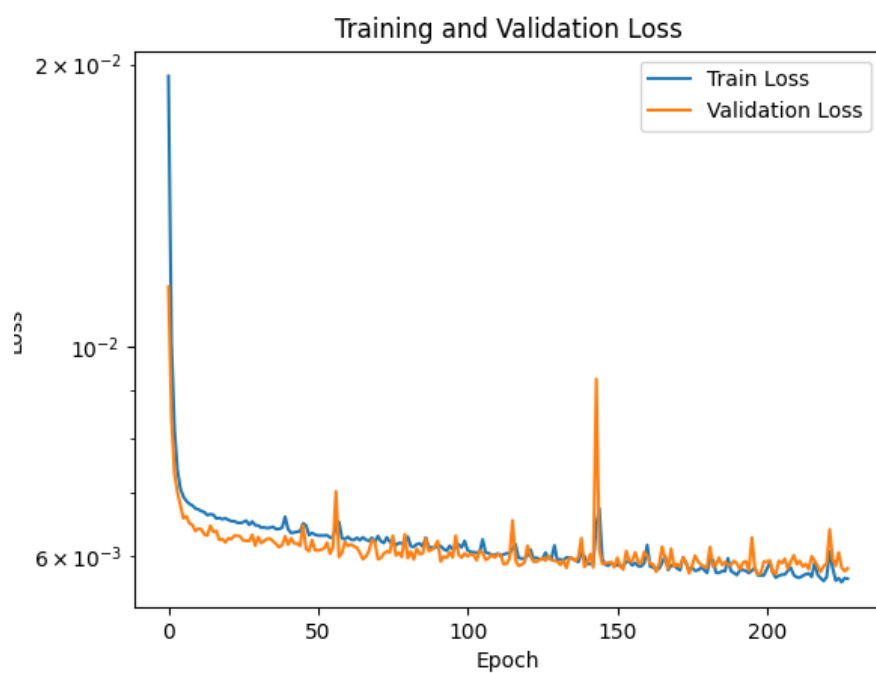


図 6.30: 学習曲線

- `models/dmodels/3000_fineAEdeephmodel_20250116_dwhite.pth`

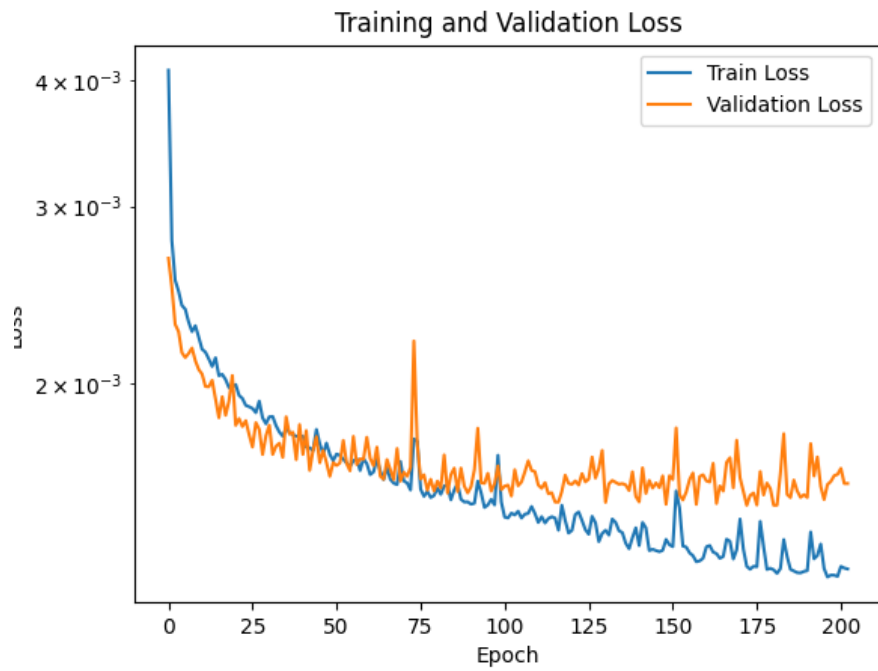


図 6.31: 学習曲線

- `models/rmodels/3000_fineAEdeephmodel_20250116_rwhite.pth`



図 6.32: 学習曲線

- `models/dmodels/2000_fineAEdeepmodel_20250116_ddark.pth`

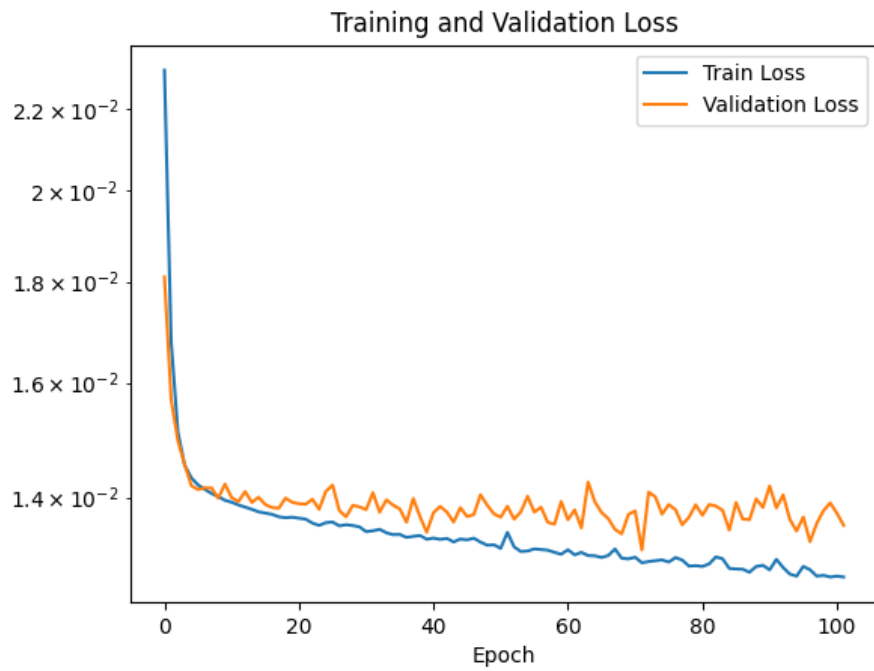


図 6.33: 学習曲線

- `models/rmodels/2000_fineAEdeepmodel_20250116_rdark.pth`



図 6.34: 学習曲線

- `models/dmodels/2000_fineAEdeepmodel_20250116_dlight.pth`

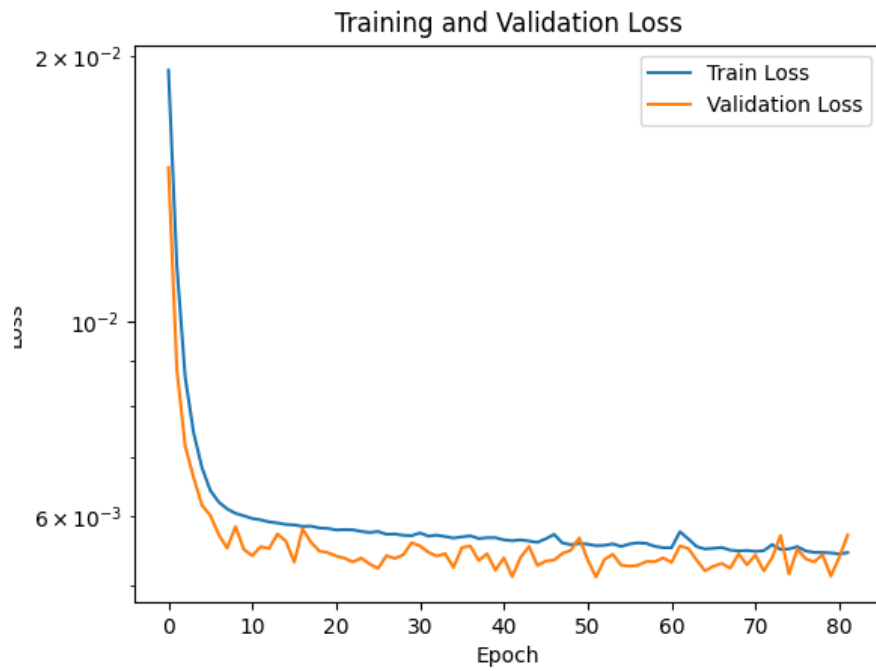


図 6.35: 学習曲線

- `models/rmodels/2000_fineAEdeepmodel_20250116_rdark.pth`

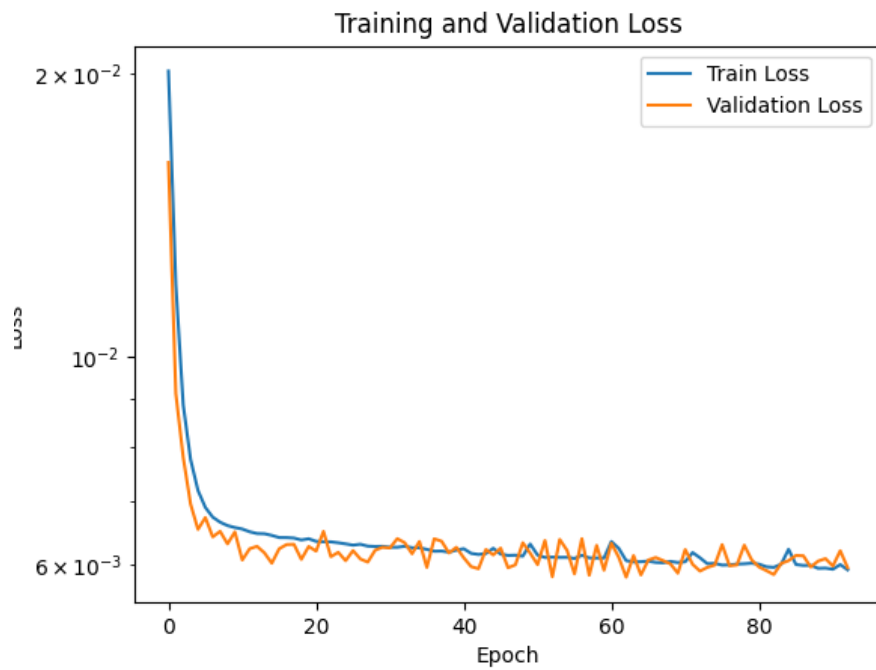


図 6.36: 学習曲線

- `models/dmodels/2000_fineAEdeepmodel_20250116_dwhite.pth`

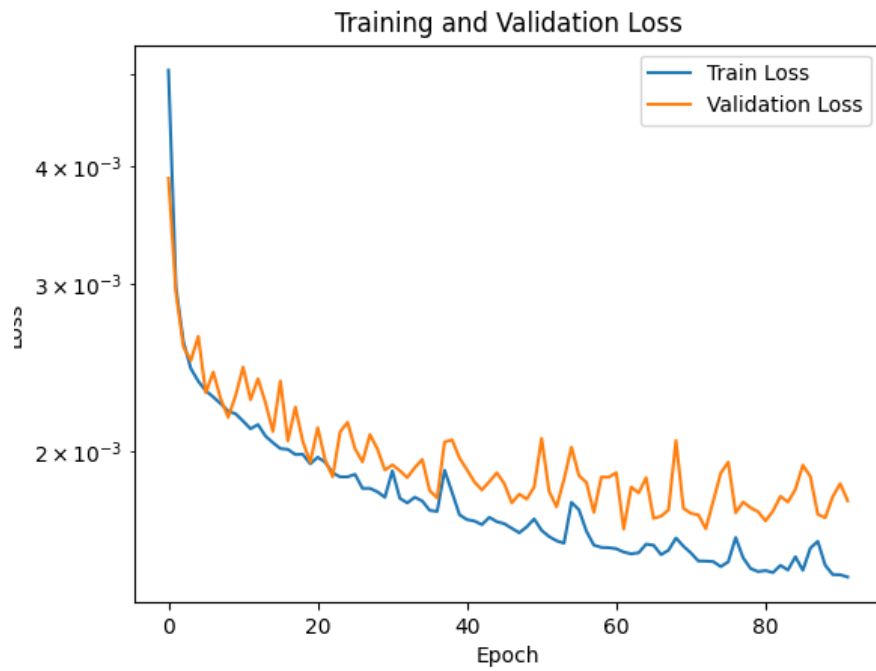


図 6.37: 学習曲線

- `models/rmodels/2000_fineAEdeepmodel_20250116_rwhite.pth`

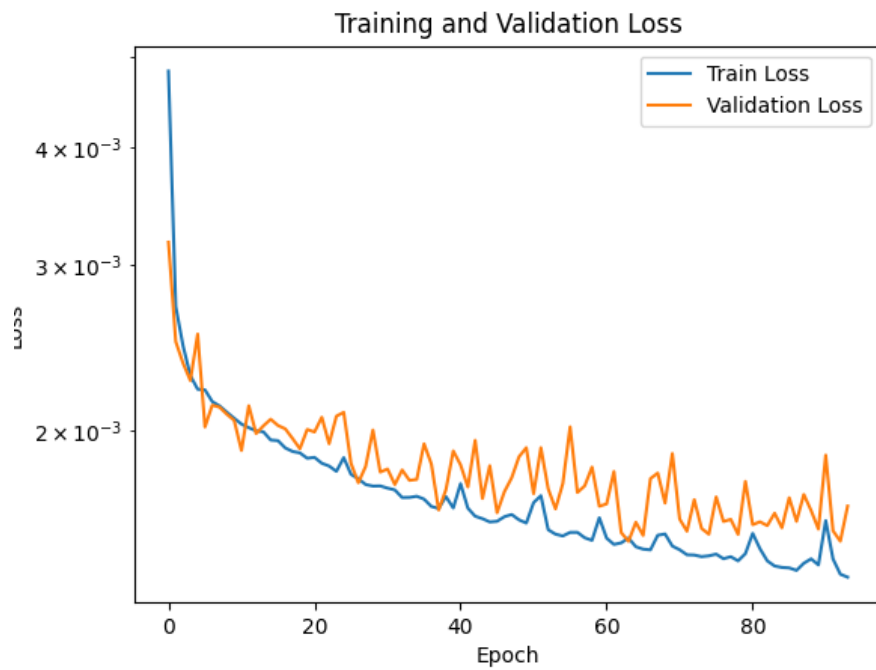


図 6.38: 学習曲線

- `models/dmodels/5000_fineAEdeepmodel_20250116_ddark.pth`



図 6.39: 学習曲線

- `models/rmodels/1000_fineAEdeepmodel_20250116_rdark.pth`

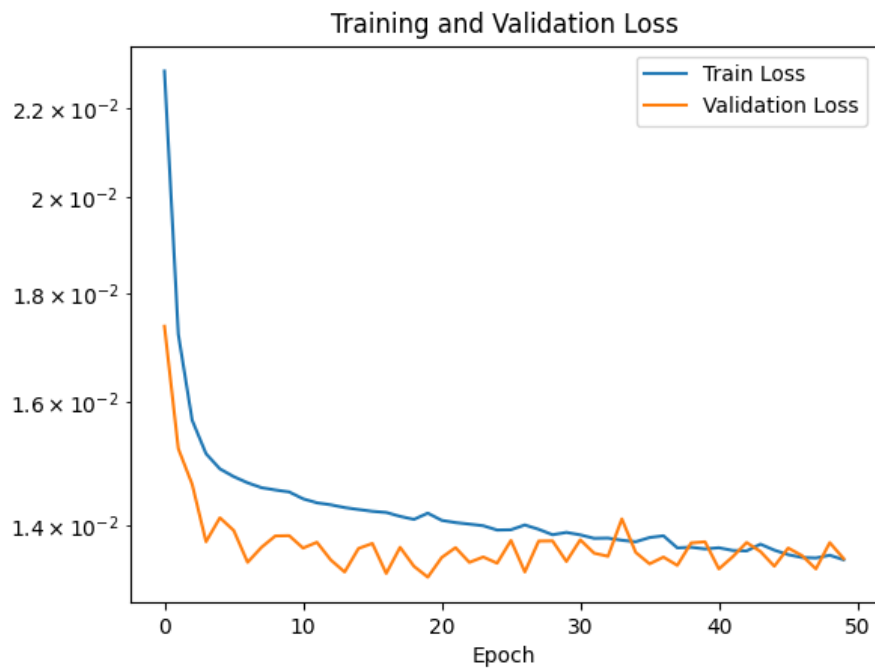


図 6.40: 学習曲線

- `models/dmodels/1000_fineAEdeepmodel_20250116_dlight.pth`

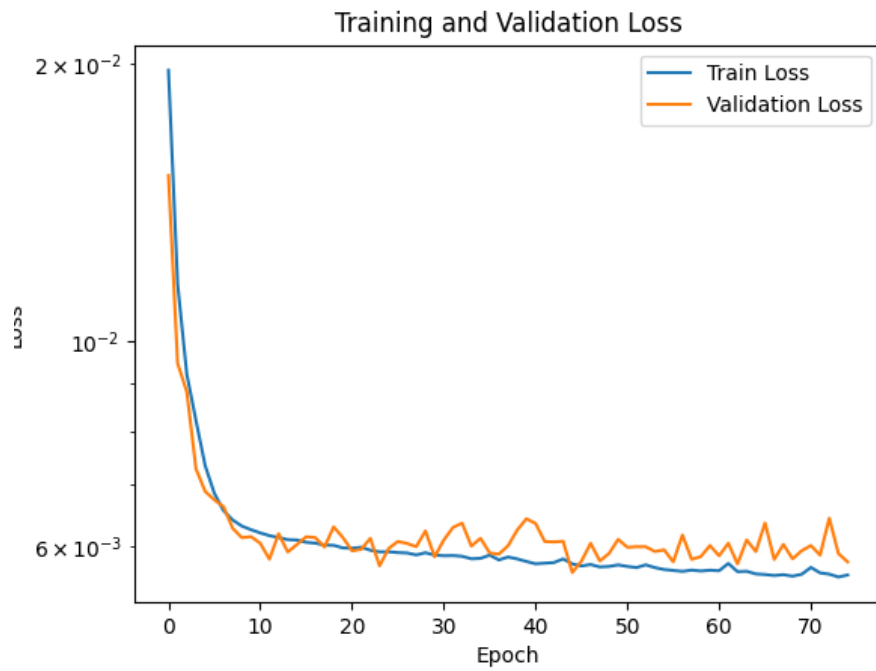


図 6.41: 学習曲線

- `models/rmodels/1000_fineAEdeepmodel_20250116_rdark.pth`

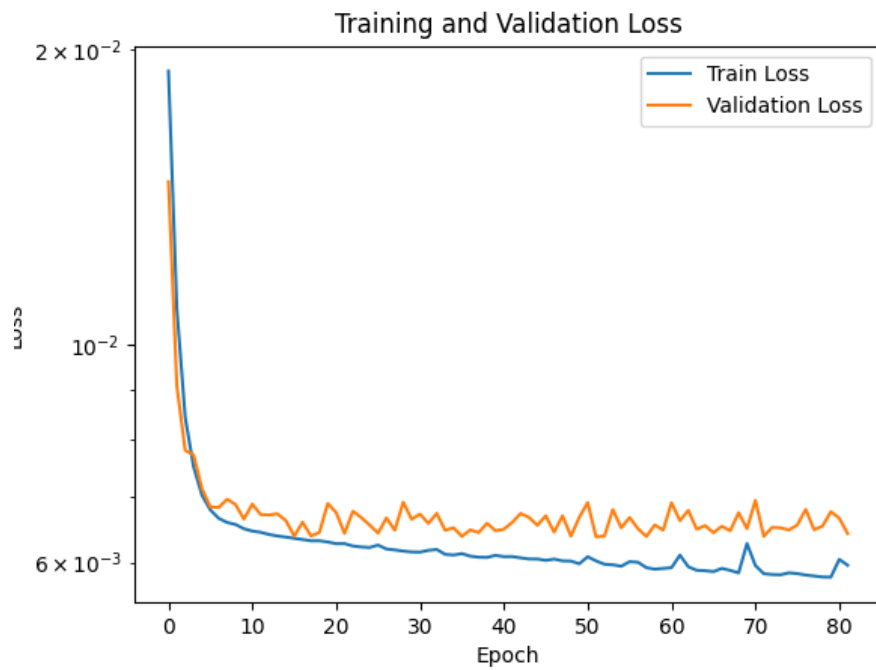


図 6.42: 学習曲線

- `models/dmodels/1000_fineAEdeepmodel_20250116_dwhite.pth`



図 6.43: 学習曲線

- `models/rmodels/1000_fineAEdeepmodel_20250116_rwhite.pth`

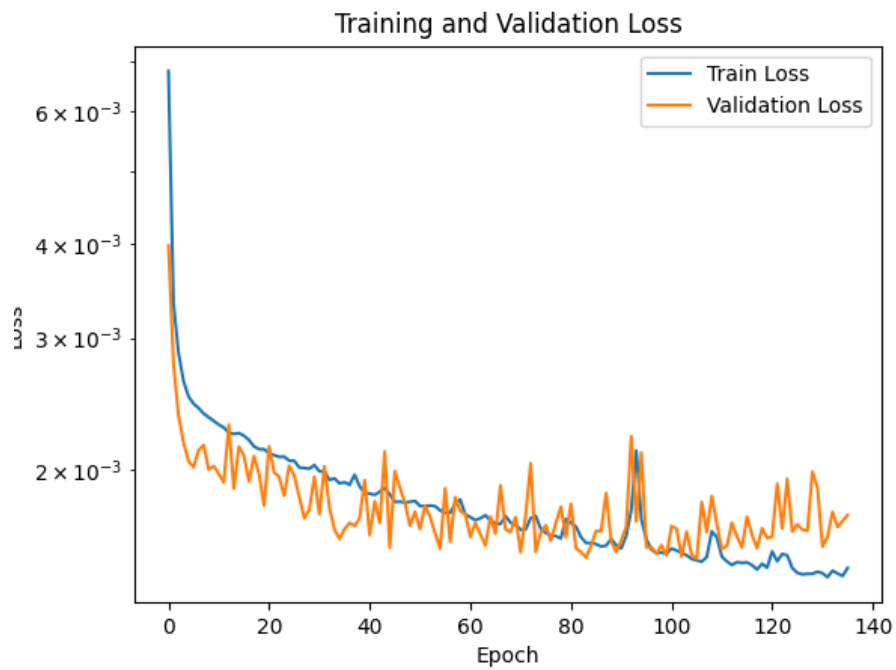


図 6.44: 学習曲線

- `models/dmodels/100_fineAEdeepmodel_20250116_ddark.pth`

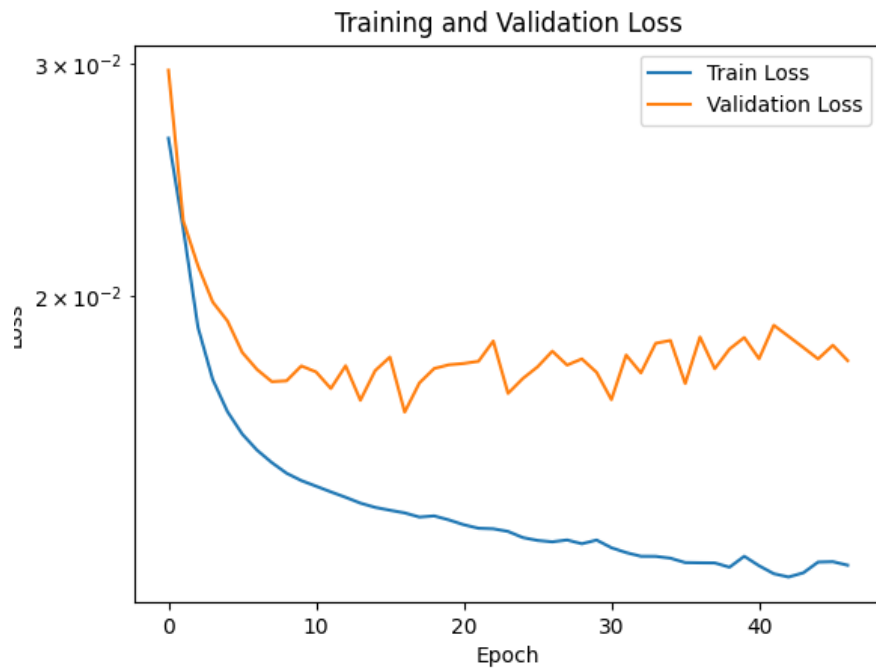


図 6.45: 学習曲線

- `models/rmodels/100_fineAEdeepmodel_20250116_rdark.pth`



図 6.46: 学習曲線

- `models/dmodels/100_fineAEdeepmodel_20250116_dlight.pth`

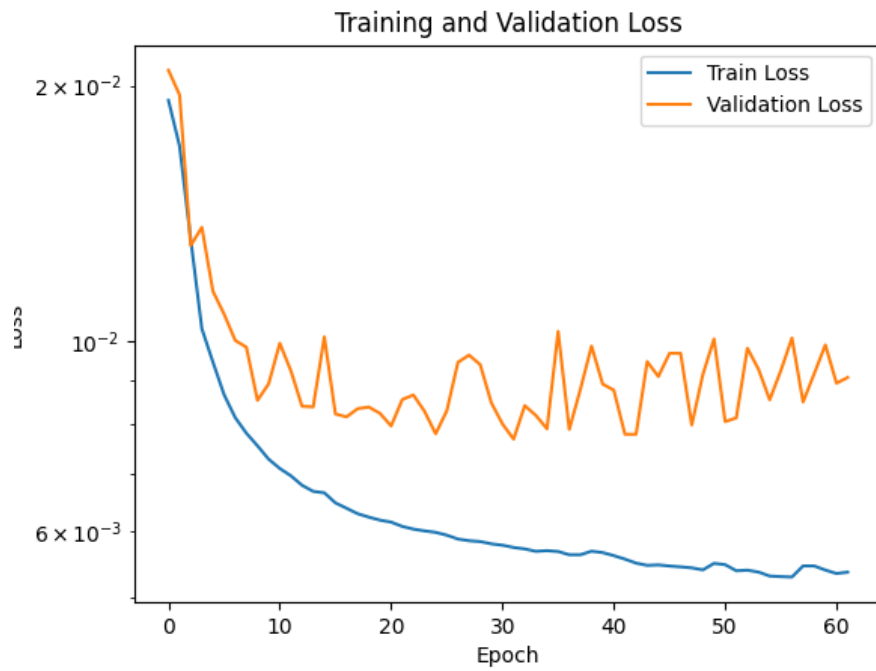


図 6.47: 学習曲線

- `models/rmodels/100_fineAEdeepmodel_20250116_rdark.pth`



図 6.48: 学習曲線

- `models/dmodels/100_fineAEdeepmodel_20250116_dwhite.pth`

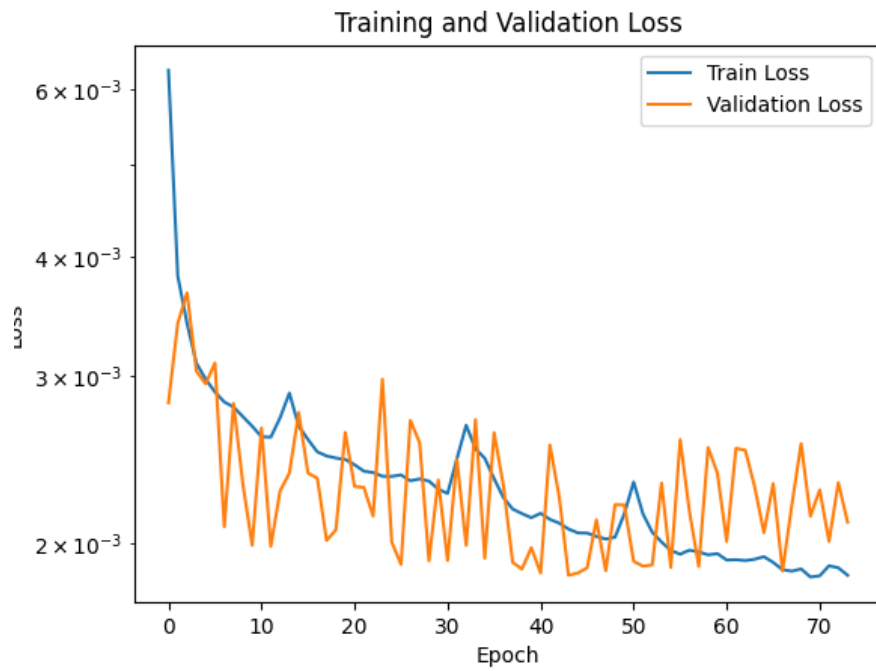


図 6.49: 学習曲線

- `models/rmodels/100_fineAEdeepmodel_20250116_rwhite.pth`

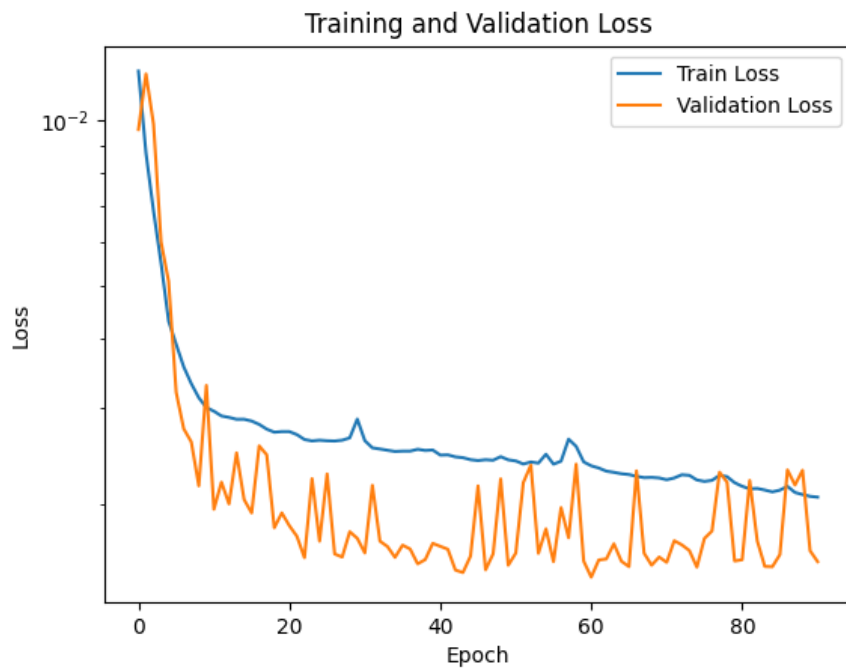


図 6.50: 学習曲線

6.7 データ

データは GitHub で公開: <https://github.com/dreamana/master-s-thesis>
画像とモデルは README に Google Drive リンクとして記載されている.

参考文献

- [1] 梅津魁秀. 深層学習での物体検出によるバードストライク調査プログラムの作成, 2023. 山形大学大学院理工学研究科数理科学分野修士論文, 未公開資料.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [5] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.
- [6] Fuzhen Zhuang, Zhiwei Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive review on transfer learning. *Proceedings of the IEEE*, 109(1):43–76, 2020.
- [7] Peter Bruce, Andrew Bruce, and Peter Gedeck. データサイエンスのための統計学入門第 2 版 予測、分類、統計モデリング、統計的機械学習と R/Python プログラミング. オライリー・ジャパン, 東京, 2020.
- [8] 小枝正直, 上田悦子, and 中村恭之. *OpenCV による画像処理入門改訂第 2 版*. オーム社, 東京, 2020.
- [9] Mlfoundations. Openclip: An open source implementation of openai’s clip. https://github.com/mlfoundations/open_clip, 2023. Accessed: 2024-12-25.
- [10] Taisei Yamana. Resnet シリーズの pytorch の公式実装コードの解説. <https://qiita.com/TaiseiYamana/items/b3e97da112d912c66563>. 最終更新日: 2021 年 10 月 21 日, Accessed: 2024-12.